

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 11:20:02
Файлов исходного кода: 82
Суммарный объём кода: 1.16 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасика [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
+env:
+  CI: false
+  GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```


- ```
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
 done
 convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density  
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| \public/export/full\_code\_all\_pages  
 echo "| PNG-страниц | \$(ls tmp/export-pages.  
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7

```
 steps:
 - name: Checkout
 - uses: actions/checkout@v4
 + uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 - uses: actions/setup-node@v4
 + uses: actions/setup-node@v7
 with:
 - node-version: "22"
 + node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
 } >> "$GITHUB_STEP_SUMMARY"

 - name: Upload artifacts (PDF + TXT)
 - uses: actions/upload-artifact@v4
 + uses: actions/upload-artifact@v7
 with:
 name: full-code-export
 path: |
@@ -107,7 +107,7 @@ jobs:
 retention-days: 30

 - name: Upload artifacts (PNG-страницы)
 - uses: actions/upload-artifact@v4
 + uses: actions/upload-artifact@v7
 with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png

`index.html`
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
```

```
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
```

```
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```

```
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
```

```
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
```

```
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
```

```
import { chapters } from '../data/content';
```

```
import { Navbar } from '../components/Navbar';
```

```
import { Sidebar } from '../components/Sidebar';
```

```
import { MobileToc } from '../components/MobileToc';
```

```
import { ChapterView } from '../components/ChapterView';
```

```
import { ChapterNav } from '../components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
  <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

  {activeTab === "guide" ? (
    <>
      <MobileToc
        selectedChapterId={activeChapterId}
        setSelectedChapterId={goTo}
        currentTitle={activeChapter.title}
        index={index}
        total={chapters.length}
      />

      <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
        {/* Сайдбар только на десктопе – на мобильном скрывается */}
        <div className="hidden lg:block lg:w-80 shrink-0" >
          <Sidebar selectedChapterId={activeChapterId} />
        </div>

        <main id="main-content" className="flex-1 m-0 p-4" >
          {/* Шапка главы с быстрыми переходами */}
          <div className="flex items-center justify-between" >
            <div className="min-w-0" >
              <div className="text-[10px] uppercase" >
                {activeChapter.category || "Универсальное пособие"}
              </div>
              <h2 className="text-base sm:text-xl font-medium" >
                {activeChapter.title}
              </h2>
            </div>
            <ChapterNav prev={prev} next={next} index={index} />
          </div>

          <div className="h-1 w-full bg-slate-800 rounded" >
            <div
              className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
              style={{ width: `${progress}%` }}
            />

```

Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/Sim

import { SimulatorHub } from "../components/SimulatorHub

+import { PythonCompiler } from "../components/PythonComp

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" |

+ const [activeTab, setActiveTab] = useState<"guide" |

const [activeChapterId, setActiveChapterId] = useSta

const [modalVizId, setModalVizId] = useState<string

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

- ) : (

+ ) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+ ) : (

+ <PythonCompiler />

}}

<SimulatorModal vizId={modalVizId} onClose={() =>

```
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-slate-800">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-compiler-btn"
 onClick={() => setActiveTab('compiler')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold text-sm duration-200 whitespace-nowrap ${
 activeTab === 'compiler'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Компилятор
 </button>
 </div>
</div>
```

```

 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};
},

```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >
      <Cpu className="w-4 h-4" /> Тренажёры

```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index-gz.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index-gz.js`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL, {
      module as PyodideModule,
    }).then((module) => module.loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}
```

```

    }
  }, [code, stdin, running, runtimeReady]);

const reset = () => {
  setCode(STARTER_CODE);
  setStdin("");
  setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
  <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
    <div className="max-w-6xl mx-auto">
      <div className="flex flex-col sm:flex-row sm:items-center">
        <div>
          <div className="inline-flex items-center gap-2">
            <Terminal className="w-4 h-4" /> Боковая панель
          </div>
          <h2 className="text-2xl sm:text-3xl font-extrabold">
            <p className="text-slate-400 mt-2 max-w-2xl">
              Пишите небольшой код и запускайте его прямо в браузере,
              который переводит CPython в WebAssembly.
            </p>
          </div>
          <a
            href="https://pyodide.org/"
            target="_blank"
            rel="noreferrer"
            className="inline-flex items-center gap-1.5"
          >
            Как это работает <ExternalLink className="w-4 h-4" />
          </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
          <section className="rounded-2xl border border-slate-200 p-4">
            <div className="flex flex-wrap items-center gap-2">
              <div className="flex items-center gap-2">
                <span className="w-2.5 h-2.5 rounded-full bg-slate-200">

```

```

        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 p-4" >
        <label className="block px-4 pt-3 text-slate-500" >
            Ввод для input() • по одной строке на клавишу
        </label>
        <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border border-slate-800"
            style={{border: '1px solid #333'}}
        />
    </div>
</section>

<section className="rounded-2xl border border-slate-800 p-4" >
    <div className="flex items-center justify-between" >
        <div className="flex items-center gap-2" >
            <Terminal className="w-4 h-4 text-emerald-500" />
        </div>
        <span className={`inline-flex items-center gap-2`} >
            {runtimeReady && <CheckCircle2 className="text-emerald-500" />}
            {runtimeReady ? "готов" : "не загружен"}
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4" >
        {output}
    </pre>
    <div className="border-t border-slate-800 p-4" >
    </div>
</section>
</div>

```

```

+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
+  const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+       <div className="max-w-6xl mx-auto">
+         <div className="flex flex-col sm:flex-row sm:items-center">
+           <div>
+             <div className="inline-flex items-center gap-2">
+               <Terminal className="w-4 h-4" /> Боковая панель
+             </div>
+             <h2 className="text-2xl sm:text-3xl font-extrabold">
+               <p className="text-slate-400 mt-2 max-w-2xl">
+                 Пишите небольшой код и запускайте его прямо в браузере,
+                 который переводит CPython в WebAssembly.
+               </p>
+             </div>
+             <a
+               href="https://pyodide.org/"
+               target="_blank"
+               rel="noreferrer"
+               className="inline-flex items-center gap-1.5"
+             >
+               Как это работает <ExternalLink className="text-slate-400" />
+             </a>
+           </div>
+
+           <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+             <section className="rounded-2xl border border-slate-200 p-4">
+               <div className="flex flex-wrap items-center gap-2">
+                 <div className="flex items-center gap-2">
+                   <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
+                     <span className="text-sm font-bold text-slate-400">
+                       <span className="text-[11px] text-slate-400">
+                         </div>
+                   <div className="flex items-center gap-2">
+                     <button
+                       type="button"
+                       onClick={reset}

```

```

+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-transparent border-slate-800 p-2"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4">
+         <div className="flex items-center justify-between">
+             <div className="flex items-center gap-2">
+                 <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+             </div>
+             <span className={`inline-flex items-center gap-2`}
+                 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+                 {runtimeReady ? "готов" : "не загружен"}}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4">
+         </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+         <Info className="w-4 h-4 shrink-0 text-indigo-500 font-mono" />
+         <span>Ctrl или Cmd + Enter запускает код. Имя файла не обязательно</span>
+     </div>
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-2">
+         <Info className="w-4 h-4 shrink-0 text-amber-500 font-mono" />
+         <span>Имя файла обязательно</span>
+     </div>
+ </div>

```

```
    },
    server: {
      host: "0.0.0.0",
      port: 5173,
      strictPort: true,
      // предпросмотр может открываться через внешний про
      allowedHosts: true,
    },
    preview: {
      host: "0.0.0.0",
      port: 4173,
      strictPort: true,
      allowedHosts: true,
    },
  },
});
\\
```

Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 },
}
```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
  - "Душные" детали (код, формулы, сложные доказательства)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <title> Дискретка для тупых – Интерактивный гид по дискретной математике
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
 "majorCapabilities": [
 "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
]
}
```

## Универсальное пособие • полный исходный код

```

 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

# algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в о

| Команда              | Шаг       | Результат           |
|----------------------|-----------|---------------------|
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |

## Универсальное пособие • полный исходный код

-----  
известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).  
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`.  
\* **Реестр интерактивов** — `src/components/vizRegistry` и для панели под главой, и для модальки из подсказки.

### ## Структура

...

|                    |                                           |
|--------------------|-------------------------------------------|
| src/               |                                           |
| App.tsx            | — оболочка, привязка глав к визуализации  |
| components/        | — интерактивные симуляторы (Действия)     |
| data/              | — контент пособия (главы/билеты)          |
| scripts/export/    | — пайплайн код → txt → png → pdf          |
| public/export/     | — сгенерированные TXT и PDF (обновляются) |
| .github/workflows/ | — автоматизация                           |

...

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

#### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и классы.
- \* Подзаголовки: Используют тег `

### `.

#### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика — параграфы `

`.

#### ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

---

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

|        |                                |                             |
|--------|--------------------------------|-----------------------------|
| Билет  | Тема                           | Статус                      |
| ---    | ---                            | ---                         |
| **1**  | Дерево отрезков (Segment Tree) | Главы нет. К                |
|        | ит вхолостую.                  |                             |
| **7**  | Планарность и раскраска графов | Номерной глав               |
|        | огии**.                        |                             |
| **11** | Мосты в графах                 | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы          | Номерной главы нет;         |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- \* `stack-dfs` → `StackViz.tsx` – \*\*Стек и DFS\*\*
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – \*\*Очередь
- \* `heap-beam-search` → `HeapViz.tsx` – \*\*Куча и лучевой
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – \*\*Дер
- \* `dynamic-programming` → `DPVisualizer.tsx` – \*\*Динами

---

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

|                                        |              |             |
|----------------------------------------|--------------|-------------|
| Глава                                  | id           | Комментарий |
| ---                                    | ---          | ---         |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e |             |

## Универсальное пособие • полный исходный код

|    |                                      |   |   |   |  |
|----|--------------------------------------|---|---|---|--|
| 17 | 17. Графы. Алгоритм Джонсона         | - | - | - |  |
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 | - | - |  |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 | - | - |  |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 | - | - |  |
| 21 | 21. Префикс-функция (π), КМП         | 4 | - | - |  |
| 22 | 22. Z-функция                        | 4 | - | - |  |
| 23 | 23. Алгоритм Ахо–Корасик             | 2 | - | - |  |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |  |
| -  | Обзор: Кратчайшие пути и Графы       | - | - | - |  |
| -  | Алгоритмы на карте                   | - | - | - |  |
| -  | Эйлер: Путь ≠ Цикл                   | - | - | - |  |
| -  | Мосты в графах: код + визуализация   | 1 | - | - |  |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)  
это 5 готовых компонентов, которые сейчас просто не...
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш...
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум inli...
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],
```



```

const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

=====

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

-----

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 KB

-----

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

---

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
 <div className="min-h-screen bg-slate-950 text-slate
```

```
 <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
 {activeTab === "guide" ? (
```

```
 <>
```

```
 <MobileToc
```

```
 selectedChapterId={activeChapterId}
```

```
 setSelectedChapterId={goTo}
```

```
 currentTitle={activeChapter.title}
```

```
 index={index}
```

```
 total={chapters.length}
```

```
 />
```

```
 <div className="flex flex-col lg:flex-row max
```

```
 {/* Сайдбар только на десктопе – на мобильном
```

```
 <div className="hidden lg:block lg:w-80 shr
```

```
 <Sidebar selectedChapterId={activeChapter
```

```
 </div>
```

```
 <main id="main-content" className="flex-1 m
```

```
 {/* Шапка главы с быстрыми переходами */}
```

```
 <div className="flex items-center justify
```

```
 <div className="min-w-0">
```

```
 <div className="text-[10px] uppercase
```

```
 {activeChapter.category || "Универс
```

```
 </div>
```

```
 <h2 className="text-base sm:text-xl f
```

```
 {activeChapter.title}
```

```
 </h2>
```

```
 </div>
```

```
 <ChapterNav prev={prev} next={next} ind
```

```
 </div>
```

```
 <div className="h-1 w-full bg-slate-800 r
```

```
 <div
```

---

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N}])(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);
 }

 if (m.index > last) frag.appendChild(document.createElement("div"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("div"));
 if (textNode.parentNode?.replaceChild(frag, textNode)) {
 //
 }
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {
 annotateTerms(el);
 } catch {
 //
 }
 };
 useLayoutEffect(run, []);
}

```

```
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
 opacity: 0;
 }
}
```

```
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
 font-size: 1.25rem !important;
 }
}
```

```
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
 }
}
```

---

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
```

```
export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]
```



```
<p class="text-xl font-mono text-white">
 двоичная куча. Пара – точка на декартовой
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
 платит $\approx n \cdot \log n$ сравнений, counting по ма
 точка вставляется спуском от корня: $x \leq$ уз
 единственно. Весь процесс – в симуляторе под
 Merge / Erase по шагам, разборы граблей
```

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 Й
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
 х». Спуск от корня: узел, который вместе
 скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
 аг – одно сравнение: чей у у корня больше,
 сохраняет порядок x). Итого $O(h)$ сравнений
```



```

 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">ОБЩЕЕ ПОНЯТИЕ</p>
 <p class="text-emerald-300 font-bold">О(н)</p>
 <p class="text-slate-400 text-xs mt-1">...</p>
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">ОБЩЕЕ ПОНЯТИЕ</p>
 <p class="text-white font-bold">О(н)</p>
 <p class="text-slate-400 text-xs mt-1">...</p>
 </div>
 </div>
</div>

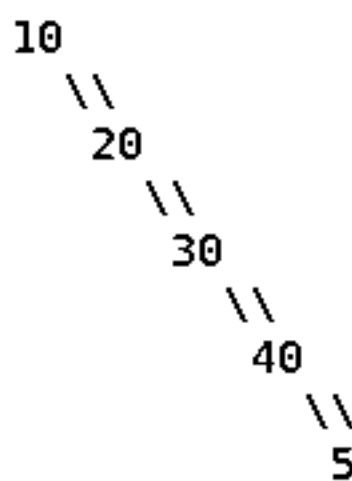
```

<!-- 1. Зачем -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border">
 <h3 class="text-lg font-bold text-white mb-3">1. Зачем?</h3>
 <p class="text-slate-300 text-sm mb-4">В обиходе, когда мы говорим о деревьях, то обычно по возрастанию — и дерево вырождается в список.
 <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto">вставили 10, 20, 30, 40, 50

```



высота = 5, поиск 50 = 5 сравнений  
это уже не дерево, а односвязный список

```

 50</pre>
 <p class="text-slate-300 text-sm mt-4">Есть два способа вставить элемент в бинарное дерево.
 <ul class="list-disc list-inside text-slate-300">
 1. Поиск
 каждой вставки чинят дерево, чтобы оно оставалось сбалансированным.
 2. Вставка
 именно быть кривым. Но каждый раз, когда мы вставляем элемент,

```

обход слева направо ДО:        A 20 B 40 C

обход слева направо ПОСЛЕ:    A 20 B 40 C        ← тот же сам

```
<div class="grid grid-cols-1 md:grid-cols-2">
```

```
 <div class="bg-slate-900 rounded-lg p-4">
```

```
 <p class="text-emerald-400 font-bol">
```

```
 <p class="text-slate-300 text-xs">П
```

сла поворотов – сломать его поворотами нево.

```
 </div>
```

```
 <div class="bg-slate-900 rounded-lg p-4">
```

```
 <p class="text-amber-400 font-bol">
```

```
 <p class="text-slate-300 text-xs">П
```

одно и то же место, поэтому В просто перев

```
 </div>
```

```
</div>
```

```
</div>
```

```
<!-- 4. Три случая -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord">
```

```
 <h3 class="text-lg font-bold text-white mb-3">
```

```
 <p class="text-slate-300 text-sm mb-4">Обоз
```

узел, который поднимаем, <span class="font

>g</span> – дед (родитель родителя). Смотри

```
<div class="overflow-x-auto -mx-2 px-2">
```

```
 <table class="w-full text-xs sm:text-sm">
```

```
 <thead>
```

```
 <tr class="text-left text-slate">
```

```
 <th class="py-2 pr-3 font-b">
```

```
 <th class="py-2 pr-3 font-b">
```

```
 <th class="py-2 pr-3 font-b">
```

```
 <th class="py-2 font-bold">
```

```
 </tr>
```

```
 </thead>
```

```
 <tbody class="text-slate-300">
```

```
 <tr class="border-b border-slat">
```

```
 <td class="py-3 pr-3 font-b">
```

```
 <td class="py-3 pr-3">деда
```

20                      40                      40

было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!</p>

`<p class="text-slate-400 text-xs mt`  
мортизации не получится.</p>

<div class="bg-slate-950 rounded-lg p-4

**<p class="text-emerald-400 font-bol**

```

<pre class="overflow-x-auto text-[1

```

$$\begin{array}{c}
 20 \\
 / \quad \backslash \\
 40 \quad 60 \\
 / \quad \backslash \\
 20 \quad A
 \end{array}
 \rightarrow
 \begin{array}{c}
 20 \\
 / \quad \backslash \\
 20 \quad 60 \\
 / \quad \backslash \\
 A \quad 40
 \end{array}
 \rightarrow
 \begin{array}{c}
 A \\
 / \quad \backslash \\
 40 \quad 60
 \end{array}$$

было: линия из 3 узлов

стало: кустик высоты 2

глубина ветки упала вдвое!</pre>

<p class="text-slate-400 text-xs mt

em. </p>

<p class="text-slate-300 text-sm mt-4"><spa

з. змейка (zig-zag) – крути СНИЗУ вверх.</p>

<!-- 6. Полный пример -->

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
```

### class="text-lq font-bold text-white mb-

<p class="text-slate-300 text-sm mb-4">Дере

3 шага, а потом делаем `Splay(10).`

```
class="bg-slate-950 p-4 rounded-lg ove
```

0 leading-relaxed">старт                      после Zi

глубина 3 → 1

**10 в корне**

```
<!-- 8. Операции -->
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-slate-700">
 <h3 class="text-lg font-bold text-white mb-4">Операции
 <p class="text-slate-300 text-sm mb-4">Красится в корне.</p>
 <div class="space-y-2 text-sm">
 <div class="bg-slate-900 rounded-lg p-3">
 — снимаем последний узел на пути (соседний по значению)
 <div class="bg-slate-900 rounded-lg p-3">
 — Spl просто отрезаем ссылки.</div>
 <div class="bg-slate-900 rounded-lg p-3">
 — sp</div>
 <div class="bg-slate-900 rounded-lg p-3">
 — Spl него не будет правого сына — туда и вешаем l
 </div>
 </div>
 </div>

<!-- 9. Код -->
<details class="bg-slate-900/40 rounded-lg border-2 border-slate-700">
 <summary class="p-4 font-bold text-slate-400 open:border-b border-slate-700/50">
 Код: rotate + splay + find (нажми, что
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto">// узел: { key, left, right, parent }
```

 </div>
</div>

// один поворот: x встаёт на место своего родителя

```
function rotate(x) {
```

```
const p = x.parent, q = p.parent;
```

```
if (p.left == x) {
```

```
p.left = x.right;
```

```
// правый поворот
```

```

 }
 return last ? splay(last) : null; // splay делает
}

```

```

 <p class="text-xs text-slate-400">Вся р
ate(x); против <span class="font-mon
т свою оценку.</p>
 </div>
</details>

```

```

<!-- 10. Плюсы/минусы -->

```

```

<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
 <div class="bg-emerald-950/30 rounded-xl p-5">
 <p class="text-emerald-300 font-bold mb-2">Плюсы
 <ul class="list-disc list-inside text-slate-300">
 Простой код: нет высот, цветов,
 Меньше памяти, чем у AVL и красн
 Сам подстраивается под «горячие»
 split / merge получаются почти

 </div>
 <div class="bg-rose-950/30 rounded-xl p-5">
 <p class="text-rose-300 font-bold mb-2">Минусы
 <ul class="list-disc list-inside text-slate-300">
 Нет гарантии на отдельную
 Не годится для реального времени
 Дерево меняется даже при чтении
 Постоянные записи в память при

 </div>
</div>

```

```

<!-- 11. Вопросы -->

```

```

<details class="bg-slate-900/40 rounded-lg border-b
 <summary class="p-4 font-bold text-slate-400">
 open:border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">

```

```


 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоугольника
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в центре.
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```



```

<div class="bg-slate-900 p-4 rounded-lg mb-12">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-1">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-1">
 получить опыт, нужна работа". Алгоритм выявл
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic
 <p class="text-xl font-mono text-white"
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутое",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic

```

```

 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отры
 </div>
 <p class="text-slate-300 text-sm mb
е сходится нечетное число линий, значит, из
а везде должно быть четное число (зашел-выш
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic
 <p class="text-xl font-mono text-white"
 d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под

```

```

 единую сеть.</p>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 19
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Остовное дерево
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Остовное дерево
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Остовное дерево — это подграф, который
 <p class="text-xl font-mono text-white">которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ★ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 i]), который одновременно является её суффиксом.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс и суффикс)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 идишь "АБРА". Если ты ошибешься при поиске
 концовка "АБРА" совпадает с началом "АБРА"
 мя перепроверок!</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код (Скрыто)
 </details>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
],
 meta: {
 title: "Алгоритм Кнута-Морриса-Пратта (КМП)",
 description: "Алгоритм Кнута-Морриса-Пратта (КМП) для поиска подстрок.",
 category: "Строки",
 tags: ["Алгоритм", "КМП", "Подстроки"],
 },
}
```

```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
</div>
<p class="text-slate-300 text-sm mb
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
</div>
</div>
<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
</div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-purple-600 text-white px-4 py-1

```

---

ФАЙЛ 18/82: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Не бы,
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic m
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounde
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
```

```

 лноту).
 </p>
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400">Теорема Эйлера
 <p>
 Связный граф содержит эйлеров цикл $\Leftrightarrow$
 2Эйлеров путь (не цикл) $\Leftrightarrow$ ровно 0
 </p>
 <p class="text-rose-400">Почему Гамильтон?
 <p>
 Задача коммивояжёра (TSP) – это взрыв мозга.
 Доказано, что любой полиномиальный алгоритм
 В общем, не парься. Просто запомни.
 </p>
 </div>
 </details>
</div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 space-y-4">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">

```



```
der-rose-500 shadow-md">
```

Аналогия: Кровеносный сосуд

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Граф – это кровеносная система. Ребра (аналогия: терали) – это не мост, живём.

Если же перерезать единственную артерию, то...

Алгоритм DFS ищет такие «критические» ребра.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border">
```

```
<summary class="p-4 font-bold text-slate-400">
```

```
order-slate-700/50">
```

Полный код на C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p>Классическая реализация – dfs (Depth-First Search) с обходом по ребрам. dfs обходит все вершины и ребра графа, находя все возможные пути от начальной вершины к целевой.
```

```
<div class="bg-slate-950 p-4 rounded-lg">
```

```
<pre class="text-xs font-mono text-slate-400">
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежности
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```
 visited[v] = true;
```

```
 tin[v] = low[v] = timer++; // устанавливаем время
```

```

title: "Планарность: K5, K3,3 и формула Эйлера",
type: "html",
content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 r

 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-blue-300 italic r
 <p class="text-3xl font-mono text-white
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-l
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-l
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!

```

```

 </div>
 </details>
 </div>
 </section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 content: `<section id="graph-articulation" class="mb-6">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Точки
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 Точки сочленения (articulation points) — это вершины графа, удаление которых увеличивает число компонент связности.

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Пример: Рассмотрим граф, состоящий из двух циклов, соединенных одной вершиной. Эта вершина является точкой сочленения.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 </div>
 <div class="p-3 bg-slate-950 rounded">
 Пример:
 <p class="text-xl font-bold text-white">Важно:
 <p class="text-xs text-slate-400">Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 </div>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Важно: Это работает только для вершин, имеющих степень больше 1. Вершины с степенью 1 (листья) не являются точками сочленения. То есть, Точка сочленения — это вершина, удаление которой увеличивает число компонент связности.
 </p>
 </div>
`
 }
]

```



```

 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20
 dedup.set(c.id, c);
 }
 });

 // Add the new ones
 if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
 }

 const finalChapters = Array.from(dedup.values());

 finalChapters.sort((a, b) => {
 const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
 const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
 return numA - numB;
 });

 export const chapters: Chapter[] = finalChapters;

 export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler",
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
 "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi",
 "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
 "string-z-func", "segment-trees", "sparse-table", "dynam
];

```

ФАЙЛ 20/82: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМ

```

import type { DemoKind } from "../components/hints/Mini

```

```

/**

```

```

 labels: ["DFS", "обход в глубину", "поиск в глубину",
 title: "DFS — обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся — откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
},
{
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта — шутлив
 quicksort.",
 formal:
 "Поиск одного элемента в отсортированном массиве -
 O(n log n). Бинарный поиск сортировку не выполн
 complexity: "поиск: O(log n); сортировкой не являетс
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение O(log n), «посмотре
 demo: "heap",
 simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

```

```

{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то обход»)",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффикснумерация, но строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень), а потом в очередь сами дети. Потом",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки"],
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошла? По суффиксной ссылке к началу строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает суффикс",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топосорт"],

```

```

 title: "Разреженная таблица (метод двоичных подъёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1, 2, ..., m.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг O(n log n), запрос O(1)",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дерево отрезков"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь массив, а листья — за отдельные куски.",
 formal: "Строится за O(n), запрос и обновление — O(log n)",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостовые"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается на две части.",
 formal: "Ребро (u,v) — мост, если low[v] > tin[u], где tin[u] — время захода в вершину u, а low[v] — время выхода из вершины v, если ребро в родителя — нет.",
 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "точки сочленения"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс.",

```





```

 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из А в В. «Дешёвый» – по с
 formal:
 "Дейкстра – неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешевое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас
 formal:
 "Корректность обычно доказывается «леммой о разре
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова

```

```

 "Матрица смежности – таблица «есть ли ребро», быс
 й, экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Спис
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: $V-1$ раз проходим по ВСЕМ рёбра
 ",
 formal:
 "После k -й итерации гарантированно верны все крат
 ось – есть отрицательный цикл.",
 complexity: " $O(V \cdot E)$ ",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд–Уоршелл: разрешаем вершину k ",

```

```

 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка $O(E \log E) + E$ операций union/find. C
 complexity: " $O(E \log E)$ ",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
 complexity: " $O(V + E)$ ",
 demo: "components",
 simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Горо
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица
 demo: "graph",
 simulator: "graph-dfs-bfs",

```

```
{
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больший отрезали дважды.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
 complexity: "запрос O(1) после O(nm) предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведё"],
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая решение A в решение B.",
 demo: "np",
},
{
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива. Нужны предподсчёты.",
 formal:
 "Суммы — префиксные суммы O(1). Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
}
```

```

{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в одну сторону, а y и z – в другую (p, x). Именно это вдвое сокращает глубину всего дерева.",
 formal:
 "Если крутить наивно (два раза поднимать x), бамбук превращается в AVL-дерево.",
 demo: "zig-zig",
 simulator: "splay-rotations",
},
{
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – правый сын h. В результате p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева.",
 demo: "zig-zag",
 simulator: "splay-rotations",
},
];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (чтобы было удобнее читать). */
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 GLOSSARY.map((e) => e.labels.map((label) => ({ label, id: e.id })))
 .flat()
 .sort((a, b) => b.label.length - a.label.length);

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
 (нельзя поехать и заработать). Он всегда работает, так как не имеет ограничений

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Аналогия 1: Позитивное планирование
 <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Аналогия 1: Позитивное планирование
 (нельзя поехать и заработать). Он всегда работает, так как не имеет ограничений
 <div id="slot-chapter-image-dijkstra" class="h-40 w-40">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Ограничения
 (нельзя поехать и заработать). Он всегда работает, так как не имеет ограничений
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь
 </div>

```

```

 <div id="slot-chapter-image-bellman"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-2">
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
 </section>`
 },
 {
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

```



```
</div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
```

```
 <h3 class="text-xl font-bold text-blue-400 mb-4">В
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
```

```
 <p class="text-sm text-blue-300 italic mb-2">0с
```

```
 <p class="text-lg font-mono text-white">Бор (Tr
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm">Позволяет найти
```

```
 -mono text-amber-300 bg-black/30 px-1 rounded">
```

```
 y.</p>
```

```
</div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
```

```
 <div class="bg-slate-800 p-6 rounded-lg border bo
```

```
 <div class="absolute -top-3 left-4 bg-slate-700
```

```
 emerald-500 shadow-md">
```

```
 Аналогия 1: Паутина (Бор)
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Представ
```

```
 мы движемся по веткам. Если нужного продолж
```

```
 ("нити страховки") в самый длинный из извест
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2
```

```
 <div class="absolute -top-3 left-4 bg-rose-900
```

```
 -500 shadow-md">
```

```
 Аналогия 2: Матёшка (Терминальные)
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Представ
```

```
 нашли и "he", потому что оно спрятано внутри
```

```
 терминальные ссылки (term_link) – прямые мос
```

```
 p>
```

```
</div>
```

```
</div>
```

```

 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
}
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-400
 <p class="text-slate-300 text-sm mb-4">Когда
 ут быть большими дробными числами (долгота
 ву от 0 до N, и уже по ним строим графы.</p>
 </div>
</div>
</section>`
}
];

```

```

 question: "Куда ведёт эйлеров путь, если нечётных
options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет — нужно удалять лишние рёбра",
],
correctIndex: 2,
explanation: "Если нечётных вершин больше 2, эйлеров
 ровно 2 нечётные или 0."
 }
],
 "bridges-code": [
 {
 question: "После DFS у нас $low[u] = 4$, а $tin[v] = 2$ ",
options: [
 "Да, так как $low[u] (4) > tin[v] (2)$ ",
 "Нет, $low[u]$ должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
correctIndex: 0,
explanation: "Условие моста: $low[u] > tin[v]$. В неориентированном графе
 },
 {
 question: "Для чего нужен массив $low[v]$?",
options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого нужно вернуться",
 "Чтобы хранить глубину рекурсии"
],
correctIndex: 1,
explanation: " $low[v]$ хранит минимальное время входа в вершину
 },
 {
 question: "Какова временная сложность алгоритма поиска мостов и точек сочленения?",
options: [
 " $O(V^2)$ — ищем мосты в квадрате",
 " $O(V + E)$ — каждый элемент один раз",

```

```
]
};
```

---

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

---

ФАЙЛ 23/82: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../..types';
```

```
export const tickets1to5: Chapter[] = [
```

```
 {
 id: "segment-trees",
 title: "1. Дерево отрезков с операциями на отрезках",
 type: "html",
 description: "Дерево отрезков – структура данных для
category: "Оптимизации и Продвинутые структуры",
 content: `
```

```
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
 <div class="space-y-8">
```

```
 <div id="intro" class="bg-slate-700/50 p-6 rounded">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-blue-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 promise (lazy).</p>
```

```
 </div>
```

```

 }
 }</pre>
</div>
</details>
</div>
</div>
</section>
\
},
{
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Сложность
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">2. Двумерная разреженная матрица (Sparse Table)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Задача
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дана матрица ST размера $L \times R$ и целое k . Для каждого запроса (l, r) найти минимальное значение среди элементов на отрезке $[l, r]$ по второму индексу.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-2">Представь, что у тебя есть набор линейки разной длины. Ты берешь две заготовленные заготовки и пытаешься покрыть ими отрезок длины 7 (перекрывая друг друга по максимуму).
 </div>
 </div>
 </div>
 </div>
`

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-blue-400">Аналогия 1: Удар катаной (Split)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Аналогия 1: Удар катаной (Split)
 <p class="text-xl font-mono text-white">
 метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 по значению X. Все, кто меньше X улетают в
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вого). Мы просто смотрим на корни: у кого Y
 как потомок.</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300 border border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);

```

### Аналогия 1: VIP-лифт

</div>

<p class="text-slate-300 text-sm mb-4">вместе с корнем дерева (VIP-статус). Если ты член клуба, то тебе не требуется времени!</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">

### Аналогия 2: Балансировка дерева

</div>

<p class="text-slate-300 text-sm mb-4">и становиться кривым как бамбук. Но как только вы станете членом клуба, вы сможете избежать этого.</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg">

<summary class="p-4 font-bold text-slate-300">Балансировка дерева</summary><div class="border border-slate-700/50">

Код (Скрыто)

</div></summary>

<div class="p-5 text-sm text-slate-300">

<pre class="bg-slate-950 p-4 rounded">

// Zig, Zag

function rotate(x) {

let p = x.parent;

if (p.left === x) { // Right rotate

p.left = x.right;

x.right = p;

} else { // Left rotate

p.right = x.left;

x.left = p;

}

}</pre>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl">

```

 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1]
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоугольника
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в центре.
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
};

```

ФАЙЛ 24/82: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМЕР: 1.2 КБ

```
import { Chapter } from '../../../types';
```







```

 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4
 rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пл
 ства шлют гонцов к ближайшему чужому царств
 </div>
 </div>
</div>
</section>`
 }
};

```

ФАЙЛ 25/82: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```

import { Chapter } from '../../../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">

```

```

<div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px; margin: 10px 0;">
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code>abc</code>
 ab: Из начала ("a") не получилось. <code>abca</code>
 abc: Опять мимо. <code>abcab</code>
 abca: О! Начало ("a") совпало. <code>abcabca</code>
 abcab: Начало ("ab") совпало. <code>abcabca</code>
 abcabc: Начало ("abc") совпало. <code>abcabcabca</code>
 abcabcd: "d" всё испортило. <code>abcabcd</code> vs <code>abcabca</code>. Конец "abcd" не совпадает с "abca".

</div>
</details>
<details class="bg-slate-900/40 rounded-lg p-5" style="border: 1px solid #ccc; margin-top: 10px;">
 <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #ccc; border-bottom: none; padding: 5px 10px 0 10px;">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; border-top: none; padding: 0 10px 10px 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="margin: 0; font-family: monospace; font-size: 0.9em;">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>

```

```
border-rose-500 shadow-md">
```

Аналогия 2: LLM Самокопиаст

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

В LLM Z-функция может применять  
[i]</code> (где <i>i</i> указывает назад на  
сама себя.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<p>Тот же пример: abcabcd</p>
```

```
<ul class="list-disc pl-5 space-y-2
```

```
<code>Z[0]</code> – не счита
```

```
<code>Z[1]</code> (bcabcd
```

```
[1] = 0</code>
```

```
<code>Z[2]</code> (cabcd
```

```

```

```
<code>Z[3]</code> (abcd<
```

? Нет, в начале 'a', тут 'd'). Совпало 3 бу

```
<code>Z[4], Z[5], Z[6]</code>
```

```

```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg l
```

```
<summary class="p-4 font-bold text-slate
```

```
-b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounde
```

```
int l = 0, r = 0;
```

```

<div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
</div>
<p class="text-slate-300 text-sm mb-4">Представ
 ет – мы падаем по суффиксной ссылке ("
</div>

<div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
 }
}

```

```

 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b0082; border-radius: 10px; box-shadow: 0 10px 15px #4b0082; position: relative;">
 <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #4b0082; border-radius: 10px; box-shadow: 0 10px 15px #4b0082; position: relative;">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Это отличный переводчик на английский (Сведения о переводе: В, то В – это NP-Полная (сосредоточена на переводе))
 </p>
 </div>
 </div>
 </div>
 </section>
}
];

```

ФАЙЛ 26/82: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">6. Графы. Представление, DFS, BFS
 </div>
 <div class="space-y-8">

```



```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Жадный алгоритм находит ближайшую
 ую вершину (например, минимальную по номеру) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300 list-disc">
 Где используется:
 Топологическая сортировка (DFS)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Косараса
 Поиск циклов и просто проверка связности

 </div>

```

```

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
 BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Концентрические волны от стартовой вершины.
 Работает через Очередь (FIFO). Продвигается по соседним вершинам.
 </p>
 <ul class="text-xs text-emerald-300 list-disc">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Алгоритм Ахо-Корасик (сборка слов)
 Алгоритм Форда-Фалкерсона / Поиск в ширину
 Двунаправленный поиск для ускорения

 </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 Аналогия BFS
 </summary>
 <div class="p-4 border border-slate-700/50">

```

```

 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Теория
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">4 цвета
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">
 1. Если граф не планарный, то он не может быть раскрашен 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Планарный граф — это граф, который можно нарисовать так, чтобы его вершины были точками, а его рёбра — непересекающимися отрезками.
 </p>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-прежнему не сливались цветами, Всегда можно всего 4 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 }

```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-indigo-400 font-bold">
```

```

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
</div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компонент
и <i>ориентированного</i> графа, в котором </p>
 </div>
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
</p>
<ul class="list-disc list-inside text-sm text-slate-300">
 Если у тебя есть цикл <code>А → Б → А</code>
 Если ты добавишь вершину Г
и четыре вершины – одна большая SCC.
 Правило: Если два цикла имеют хотя бы одну
вершину, то они образуют один цикл.


```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-emerald-500 shadow-md">
 Аналогия: Интриги в офисе
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">
 Сильная связность – это "клуб сильной связности"
ни в одной SCC. Добавить курьера Гошу, который может
йти наружу (к директору), но обратно в клуб не может.
</p>
</div>
```

```

 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и $f(u,v) > t(u,v)$.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 },

```

Представь город, где два района соединены перекрестком (удалят вершину) — районы будут **хрупкостью** системы.

Телеком: Центральный свитч

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

Душный мод: Код и корень DFS (Скрыто)

Для стартовой вершины обхода правил то выше неё прыгать некуда). Поэтому для него **больше 1** независимого ребенка

```
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;
```

```
 for (int to : adj[v]) {
```

```

{
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-white">13. Графы. Эйлеров цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Эйлеров цикл
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Эйлеров цикл — это замкнутый путь, который проходит по каждому ребру графа ровно один раз.
 <p class="text-xl font-mono text-white">Граф имеет Эйлеров цикл тогда и только тогда, когда у каждого вершины графа четная степень.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">Если у вас есть лист бумаги, на котором нарисованы всевозможные замкнутые контуры, и вы пытаетесь их все обвести, не отрывая пера, то вы сможете это сделать только в том случае, если каждый контур имеет четное количество сторон.
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <p class="text-slate-300 text-sm mb-4">Если же у вас есть граф, в котором есть вершины с нечетной степенью, то вы не сможете пройти по всем ребрам ровно один раз.
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
];

```

```

 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);
 });

```



```

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),

```

```

 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l
 <svg className="w-full h-full" viewBox="0 0 8
 {/* Edges */}
 {edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge
 const v = nodes.find((n) => n.id === edge

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 />
)
 })}
 </svg>
 </div>
 </div>

```

```

/>
<text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-even"
>
 {node.label}
</text>

{isVisited && {
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]

```

```

 Мост
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `>
 >
 {step.desc}
 </div>
 </div>
 })}
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition"
 style={{
 width: `${(currentStepIndex / Math.ma
 }}
 />
 </div>
 </div>
 </div>
</div>
</div>
);

```

```
const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start)' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1' },
 { line: 4, text: ' for (u, v, weight) in E:' },
 { line: 5, text: ' if dist[u] + weight < dist[v]' },
 { line: 6, text: ' dist[v] = dist[u] + weight' },
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл' },
 { line: 8, text: ' if dist[u] + weight < dist[v]' },
 { line: 9, text: ' return "Обнаружен отрицательный цикл"' },
 { line: 10, text: ' return dist' },
 { line: 11, text: '}' },
 { line: 12, text: '' },
 { line: 13, text: 'const graph = { nodes, edges }' },
 { line: 14, text: 'const start = "S"' },
 { line: 15, text: 'const dist = bellman_ford(graph, start)' },
 { line: 16, text: 'console.log(dist)' },
 { line: 17, text: '' },
 { line: 18, text: 'const graph = { nodes, edges }' },
 { line: 19, text: 'const start = "S"' },
 { line: 20, text: 'const dist = bellman_ford(graph, start)' },
 { line: 21, text: 'console.log(dist)' },
 { line: 22, text: '' },
 { line: 23, text: 'const graph = { nodes, edges }' },
 { line: 24, text: 'const start = "S"' },
 { line: 25, text: 'const dist = bellman_ford(graph, start)' },
 { line: 26, text: 'console.log(dist)' },
 { line: 27, text: '' },
 { line: 28, text: 'const graph = { nodes, edges }' },
 { line: 29, text: 'const start = "S"' },
 { line: 30, text: 'const dist = bellman_ford(graph, start)' },
 { line: 31, text: 'console.log(dist)' },
 { line: 32, text: '' },
 { line: 33, text: 'const graph = { nodes, edges }' },
 { line: 34, text: 'const start = "S"' },
 { line: 35, text: 'const dist = bellman_ford(graph, start)' },
 { line: 36, text: 'console.log(dist)' },
 { line: 37, text: '' },
 { line: 38, text: 'const graph = { nodes, edges }' },
 { line: 39, text: 'const start = "S"' },
 { line: 40, text: 'const dist = bellman_ford(graph, start)' },
 { line: 41, text: 'console.log(dist)' },
 { line: 42, text: '' },
 { line: 43, text: 'const graph = { nodes, edges }' },
 { line: 44, text: 'const start = "S"' },
 { line: 45, text: 'const dist = bellman_ford(graph, start)' },
 { line: 46, text: 'console.log(dist)' },
 { line: 47, text: '' },
 { line: 48, text: 'const graph = { nodes, edges }' },
 { line: 49, text: 'const start = "S"' },
 { line: 50, text: 'const dist = bellman_ford(graph, start)' },
 { line: 51, text: 'console.log(dist)' },
 { line: 52, text: '' },
 { line: 53, text: 'const graph = { nodes, edges }' },
 { line: 54, text: 'const start = "S"' },
 { line: 55, text: 'const dist = bellman_ford(graph, start)' },
 { line: 56, text: 'console.log(dist)' },
 { line: 57, text: '' },
 { line: 58, text: 'const graph = { nodes, edges }' },
 { line: 59, text: 'const start = "S"' },
 { line: 60, text: 'const dist = bellman_ford(graph, start)' },
 { line: 61, text: 'console.log(dist)' },
 { line: 62, text: '' },
 { line: 63, text: 'const graph = { nodes, edges }' },
 { line: 64, text: 'const start = "S"' },
 { line: 65, text: 'const dist = bellman_ford(graph, start)' },
 { line: 66, text: 'console.log(dist)' },
 { line: 67, text: '' },
 { line: 68, text: 'const graph = { nodes, edges }' },
 { line: 69, text: 'const start = "S"' },
 { line: 70, text: 'const dist = bellman_ford(graph, start)' },
 { line: 71, text: 'console.log(dist)' },
 { line: 72, text: '' },
 { line: 73, text: 'const graph = { nodes, edges }' },
 { line: 74, text: 'const start = "S"' },
 { line: 75, text: 'const dist = bellman_ford(graph, start)' },
 { line: 76, text: 'console.log(dist)' },
 { line: 77, text: '' },
 { line: 78, text: 'const graph = { nodes, edges }' },
 { line: 79, text: 'const start = "S"' },
 { line: 80, text: 'const dist = bellman_ford(graph, start)' },
 { line: 81, text: 'console.log(dist)' },
 { line: 82, text: '' },
 { line: 83, text: 'const graph = { nodes, edges }' },
 { line: 84, text: 'const start = "S"' },
 { line: 85, text: 'const dist = bellman_ford(graph, start)' },
 { line: 86, text: 'console.log(dist)' },
 { line: 87, text: '' },
 { line: 88, text: 'const graph = { nodes, edges }' },
 { line: 89, text: 'const start = "S"' },
 { line: 90, text: 'const dist = bellman_ford(graph, start)' },
 { line: 91, text: 'console.log(dist)' },
 { line: 92, text: '' },
 { line: 93, text: 'const graph = { nodes, edges }' },
 { line: 94, text: 'const start = "S"' },
 { line: 95, text: 'const dist = bellman_ford(graph, start)' },
 { line: 96, text: 'console.log(dist)' },
 { line: 97, text: '' },
 { line: 98, text: 'const graph = { nodes, edges }' },
 { line: 99, text: 'const start = "S"' },
 { line: 100, text: 'const dist = bellman_ford(graph, start)' },
 { line: 101, text: 'console.log(dist)' },
 { line: 102, text: '' },
 { line: 103, text: 'const graph = { nodes, edges }' },
 { line: 104, text: 'const start = "S"' },
 { line: 105, text: 'const dist = bellman_ford(graph, start)' },
 { line: 106, text: 'console.log(dist)' },
 { line: 107, text: '' },
 { line: 108, text: 'const graph = { nodes, edges }' },
 { line: 109, text: 'const start = "S"' },
 { line: 110, text: 'const dist = bellman_ford(graph, start)' },
 { line: 111, text: 'console.log(dist)' },
 { line: 112, text: '' },
 { line: 113, text: 'const graph = { nodes, edges }' },
 { line: 114, text: 'const start = "S"' },
 { line: 115, text: 'const dist = bellman_ford(graph, start)' },
 { line: 116, text: 'console.log(dist)' },
 { line: 117, text: '' },
 { line: 118, text: 'const graph = { nodes, edges }' },
 { line: 119, text: 'const start = "S"' },
 { line: 120, text: 'const dist = bellman_ford(graph, start)' },
 { line: 121, text: 'console.log(dist)' },
 { line: 122, text: '' },
 { line: 123, text: 'const graph = { nodes, edges }' },
 { line: 124, text: 'const start = "S"' },
 { line: 125, text: 'const dist = bellman_ford(graph, start)' },
 { line: 126, text: 'console.log(dist)' },
 { line: 127, text: '' },
 { line: 128, text: 'const graph = { nodes, edges }' },
 { line: 129, text: 'const start = "S"' },
 { line: 130, text: 'const dist = bellman_ford(graph, start)' },
 { line: 131, text: 'console.log(dist)' },
 { line: 132, text: '' },
 { line: 133, text: 'const graph = { nodes, edges }' },
 { line: 134, text: 'const start = "S"' },
 { line: 135, text: 'const dist = bellman_ford(graph, start)' },
 { line: 136, text: 'console.log(dist)' },
 { line: 137, text: '' },
 { line: 138, text: 'const graph = { nodes, edges }' },
 { line: 139, text: 'const start = "S"' },
 { line: 140, text: 'const dist = bellman_ford(graph, start)' },
 { line: 141, text: 'console.log(dist)' },
 { line: 142, text: '' },
 { line: 143, text: 'const graph = { nodes, edges }' },
 { line: 144, text: 'const start = "S"' },
 { line: 145, text: 'const dist = bellman_ford(graph, start)' },
 { line: 146, text: 'console.log(dist)' },
 { line: 147, text: '' },
 { line: 148, text: 'const graph = { nodes, edges }' },
 { line: 149, text: 'const start = "S"' },
 { line: 150, text: 'const dist = bellman_ford(graph, start)' },
 { line: 151, text: 'console.log(dist)' },
 { line: 152, text: '' },
 { line: 153, text: 'const graph = { nodes, edges }' },
 { line: 154, text: 'const start = "S"' },
 { line: 155, text: 'const dist = bellman_ford(graph, start)' },
 { line: 156, text: 'console.log(dist)' },
 { line: 157, text: '' },
 { line: 158, text: 'const graph = { nodes, edges }' },
 { line: 159, text: 'const start = "S"' },
 { line: 160, text: 'const dist = bellman_ford(graph, start)' },
 { line: 161, text: 'console.log(dist)' },
 { line: 162, text: '' },
 { line: 163, text: 'const graph = { nodes, edges }' },
 { line: 164, text: 'const start = "S"' },
 { line: 165, text: 'const dist = bellman_ford(graph, start)' },
 { line: 166, text: 'console.log(dist)' },
 { line: 167, text: '' },
 { line: 168, text: 'const graph = { nodes, edges }' },
 { line: 169, text: 'const start = "S"' },
 { line: 170, text: 'const dist = bellman_ford(graph, start)' },
 { line: 171, text: 'console.log(dist)' },
 { line: 172, text: '' },
 { line: 173, text: 'const graph = { nodes, edges }' },
 { line: 174, text: 'const start = "S"' },
 { line: 175, text: 'const dist = bellman_ford(graph, start)' },
 { line: 176, text: 'console.log(dist)' },
 { line: 177, text: '' },
 { line: 178, text: 'const graph = { nodes, edges }' },
 { line: 179, text: 'const start = "S"' },
 { line: 180, text: 'const dist = bellman_ford(graph, start)' },
 { line: 181, text: 'console.log(dist)' },
 { line: 182, text: '' },
 { line: 183, text: 'const graph = { nodes, edges }' },
 { line: 184, text: 'const start = "S"' },
 { line: 185, text: 'const dist = bellman_ford(graph, start)' },
 { line: 186, text: 'console.log(dist)' },
 { line: 187, text: '' },
 { line: 188, text: 'const graph = { nodes, edges }' },
 { line: 189, text: 'const start = "S"' },
 { line: 190, text: '
```

```

 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount - 1) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v, w: edge.w },
 distances: { ...dist }, previous: { ...prevDist },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v} с весом ${edge.w}.`,
 activeCodeLine: 9,
 });
 break;
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prevDist },
 log: ` Проверка завершена. Ни одно ребро не нарушило условия.`,
 activeCodeLine: 10,
 });
 }
}

setSteps(simulationSteps);

```

```

 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlaying)}
 position-colors text-white ${isPlaying ? 'bg-blue-500' : 'bg-slate-600'}
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-slate-600 text-white px-3 py-2 rounded-lg"
 s">⏮ ⏪ ⏩ ⏭ </button>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 mt-6" >
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-blue-950/20 p-6 rounded-lg" >
 <div className="absolute top-3 left-3 bg-blue-950/20 p-3 rounded-lg shadow-sm" >
 Итерация {currentStep.iteration} (⏮ ⏪ ⏩ ⏭)
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-blue-950/20 p-3 rounded-lg shadow-sm" >
 animate-pulse shadow-lg shadow-rose-600/50
 ⚠ Отрицательный цикл!
 </div>
)}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u.id);
 const vNode = nodes.find((n) => n.id === v.id);

```

```

 </g>
);
 }}}}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">Шаг 1: Инициализация</p>
 <p className="min-h-[40px] flex items-center">
 </div>
 </div>

 {/** Список смежности */}
 <div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">Шаг 2: Поиск</h4>
 <div className="space-y-2 text-sm font-mono text-white">
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2">
 {u}
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === edge.v;
 return (
 <span key={idx} className={`${px-2} ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 } ${edge.v} (${edge.w})`>

)
)
 })}
 </div>
 })}
 </div>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/** Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10 p-4 rounded-2xl">
 <div>

```



---

ФАЙЛ 29/82: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
```

```

 logs: `Достаем из очереди (${curr}).`
 });

 const neighbors = ADJ[curr];
 if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.
 });
 }
 }
 } else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
 }
}

```

```

return (
 <div className="bg-slate-900 border-2 border-slate-
 <div className="flex flex-col md:flex-row items-c
 <div>
 <h3 className="text-2xl font-bold text-white
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-blue
 title={isPlaying ? 'Пауза' : 'Воспроизведен
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-2 g
 { /* Визуал графа */ }

```

```

 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 }
}
</svg>
</div>

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-sl
 er">
 <div className="bg-slate-800 text-slate-400
 se tracking-wider">
 bfs(graph, start)

```

```

 {qId}

))
 })
</div>
<div className="mt-2 text-sm text-amber-300
ms-center justify-center">
 "{step.logs}"
</div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 30/82: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const imageMap: Record<string, { src: string; alt: string; caption: string; border: string; captionColor: string; }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 },
 floyd: {
 src: floydImg,
 alt: 'Сеть Флойда',
 caption: 'Сеть Флойда — это просто!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
};

```

```

 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» – как на Code
 * Компактный вариант живёт в шапке главы, крупный – по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }
}
```

---

ФАЙЛ 32/82: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, Loader2, Maximize2, MousePointer } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```
 useTermHints(contentRef, [chapter.id]);
```

```
 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
```

```

 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
 };

```

```

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) scheduleHide();
};

```

```

const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

```

```

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

```

```

const viz = getViz(chapter.id);

```

```

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-xs font-medium border border-gray-200 px-2 py-1.5"
 300 hover:text-white hover:border-emerald-500
 >
 {saving === "work" ? (

```



```

{viz && {
 <section id="chapter-viz" className="mt-8 scr
 <div className="flex items-center justify-b
 <h3 className="flex items-center gap-2 te
 <Sparkles className="w-4 h-4 text-indig
 Демонстрация: {viz.title}
 </h3>
 <button
 onClick={() => onOpenSimulator(chapter.
 className="flex items-center gap-1.5 te
 ate-300 hover:text-white hover:border-indig
 >
 <Maximize2 className="w-3.5 h-3.5" /> P
 </button>
 </div>
 {viz.hint && <p className="text-xs text-sla
 <viz.Component />
 </section>
 }}

 <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
/>
</>
});
};

```

```

 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 }
];

```

```

tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
activeEdge: [2,3], backEdge: null,
log: "Идём C→D. tin[D]=5, low[D]=5.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4],
tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
activeEdge: [3,4], backEdge: null,
log: "D→E. tin[E]=6, low[E]=6.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: [4,5], backEdge: null,
log: "E→F. tin[F]=7, low[F]=7.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: [5,3],
log: "Из F видим D (посещён). Обратное ребро (F,D).
activeCodeLine: [7,8,9]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: null, backEdge: null,
log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо

```

```

 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
 }, [dfsAuto]);

 const step = DFS_STEPS[dfsIdx];

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Step {step}</h4>
 <div className="flex items-center gap-1 bg-slate-800 p-1">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 disabled={dfsIdx === 0}
 className="p-1.5 hover:bg-slate-700 rounded text-white">
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${
 dfsAuto ? "bg-rose-600 text-white" : "bg-slate-800 text-white"
 }`}>
 {dfsAuto ? <><Pause className="h-3 w-3" /> : <><Play className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1);
 disabled={dfsIdx === DFS_STEPS.length - 1}
 className="p-1.5 hover:bg-slate-700 rounded text-white">
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 className="p-1.5 hover:bg-slate-700 rounded text-white">
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
)
)

```



```
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 34/82: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
],
```

```

 { line: 11, text: ' return dist // Все кратчайшие
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu

```

```

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из ПИ
 activeCodeLine: 2,
 });

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}

```

```

 if (!u) break;

```

```

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited }
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
 });

```

```

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border-2 border-indigo-300" >
 /* Шапка симулятора */
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white rounded" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300">Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-2" >
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 className="flex items-center gap-1 bg-slate-800/90 border-2 border-indigo-300 text-white rounded-2xl px-4 py-2" >
 Сброс
 </button>
 </div>
 </div>
)

```



```

 </marker>
 </defs>
 { /* Рёбра */
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.u] < edge.v;

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-check)';
 else if (isOptimal) marker = 'url(#arrow-dh-optimal)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#2e8b57' : '#444'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : isOptimal ? '1,0' : '0,0'}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#2e8b57' : '#444'}
 />
 <text
 x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2) + 15}
 fill={isChecking ? '#f59e0b' : isOptimal ? '#2e8b57' : '#444'}
 fontSize="11" fontWeight="bold" textAnchor="center"
 />
 {edge.w}
 </g>
);
 })}
 </svg>

```

```

 </div>
 </div>

 {/* Список смежности */}
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u
 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
 }`>
 <div className="flex items-center gap-2">
 <span className={`px-2 py-0.5 rounded
 te-300'}`>{u}
 →
 </div>
 <div className="flex flex-wrap gap-2">
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.v
 return (
 <span key={idx} className={`px-2 py-0.5 rounded
 isCheckingThis ? 'bg-amber-900/60' : 'bg-slate-300'
 }`>{edge.v}
)
 })}
 </div>
 </div>
)
 })}
 </div>
 </div>
);

```

```

 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60"
 <div>
 <h4 className="text-lg font-bold text-slate-400">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4"
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line;
 return (
 <div key={item.line} className={`py-1 px-2
 ${isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'}>
 {item.line}
 {item.code}
 {isActive &&

 }
 </div>
);
 })}
 </div>
 </div>
</div>
</div>
</div>
</div>
);
}

```

```
function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
 }

 generatedSteps.push({
 id: stepId++,
 n,
```



```

disabled={currentStepIndex === steps.length}
className="p-2 text-slate-400 hover:text-white"
title="Шаг вперед"
>
 <ChevronRight className="w-5 h-5" />
</button>
</div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded" >
 <h4 className="text-xs font-bold uppercase tracking-wide">DP Table</h4>
 <div className="flex flex-wrap items-center gap-2">
 {Array.from({ length: targetN }, (_, i) => i + 1).map(num => {
 const isCached = memoState[num] !== null;
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center justify-center gap-2
 ${isCurrent ? 'bg-emerald-950 border-emerald-500' : ''}
 ${isCached ? 'bg-slate-900 border-emerald-500/50' : ''}
 ${!isCurrent & !isCached ? 'bg-slate-900/50 border-slate-800' : ''}
 `>
 {num}
 {val}
 {num <= 2 && {val}}
 </div>
);
 })}
 </div>
</div>
</div>

```

```

 </button>
 </div>

 {/* Tab Content */}
 <div className="min-h-[320px] flex flex-col justify-between" >
 {activeTab === 'table' && (
 <div className="bg-slate-950 p-6 rounded-2xl border" >
 <h5 className="font-bold text-white text-sm" >

 </h5>
 <p className="text-sm text-slate-400" >
 В классической рекурсии для N={targetN} по
 вычисленное значение в таблице, мы момента
 </p>
 <div className="p-4 bg-slate-900 rounded-xl border
 t-slate-300">
 Всего шагов симуляции: {steps.length}
 <span className="text-emerald-400 font-bo
 </div>
 </div>
)}

 {activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border" >
 <div className="bg-slate-900 px-6 py-3 border" >

 </div>
 <pre className="p-6 font-mono text-sm space" >
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 p
 isActive
 ? 'bg-emerald-600/20 border-l-4

```

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Кликни на вершину
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
```



</div>

<div className="bg-slate-950 rounded-xl p-4 border  
-hidden">

<div className="absolute inset-0 bg-[radial-gra

<svg className="w-full h-[260px] z-10 relative"

{C1\_EDGES.map((e, i) => {

const u = C1\_NODES.find(n => n.id === e.u)!

const v = C1\_NODES.find(n => n.id === e.v)!

const key = `\${Math.min(e.u, e.v)}-\${Math.ma

const used = c1VisitedEdges.includes(key);

return <line key={i} x1={u.x} y1={u.y} x2={v

stroke={used ? "#6366f1" : "#475569"}

strokeWidth={used ? 4 : 2}

className="transition-all duration-300" />

}}}

{C1\_NODES.map(n => {

const isLast = c1Path[c1Path.length-1] === n

const visited = c1Path.includes(n.id);

return <g key={n.id} className="cursor-poin

{isLast && <circle cx={n.x} cy={n.y} r={11}

<circle cx={n.x} cy={n.y} r={11}

fill={isLast ? "#6366f1" : visited ? "#

stroke={isLast ? "#fff" : visited ? "#6

strokeWidth={2.5}

className="transition-all duration-200

<text x={n.x} y={n.y+4} textAnchor="middl

bel}</text>

</g>;

}}}

</svg>

<div className="absolute top-2 left-2 bg-slate-

Pe6ëp: {c1VisitedEdges.length}/{C1\_EDGES.leng

</div>

</div>

<div className={`p-3 rounded-xl border text-sm \${

c1Done && c1VisitedEdges.length === C1\_EDGES.le

```

],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане
 Это нарушит структуру дерева.'
},
{
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?
 options: [
 'Потому что он написан только для положительных чисел',
 'Он жадный и считает, что каждый следующий шаг дает
 закрытые вершины.',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты приедешь
 ля чего нужен алгоритм Беллмана-Форда.'
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?
 options: [
 'Он использует меньше памяти благодаря коммунити',
 'Он умеет работать с отрицательными весами без цип',
 'Он позволяет выполнять шаги слияния колхозов (ком',
 'Он был разработан в Токио и поэтому самый быстрый
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая ком
 лелить вычисления на GPU или многопроцессорных кл
},
{
 id: 4,
 question: 'В чем главная суть Динамического программирования?
 options: [
 'Писать код очень быстро, динамично нажимая на кл',
 'Запоминать (кешировать) результаты уже решенных

```

```

const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты</h3>
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} баллов</p>

 <div className="bg-slate-950 border border-slate-800 p-12">
 <p className="text-slate-400 text-sm uppercase">Оценка:</p>
 <p className="text-5xl font-black text-indigo-500">{score}</p>
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
 : score >= 3
 ? ' Отличный результат! Главные концепции усвоены!'
 : ' Нужно еще поработать над некоторыми темами.'
 }
 </p>
 </div>
 </div>
);
}

```

```

const isSelected = selectedOption === idx;
const isCorrect = idx === currentQ.correctA

let optionStyle = "bg-slate-800/80 hover:bg-
if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-ro
 } else {
 optionStyle = "bg-slate-800/40 border-s
 }
}

return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 te
) : isAnswered && isSelected && !isCo
 <XCircle className="w-6 h-6 text-ro
) : (
 <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-l
 {option}
 </div>
 </div>
);
}})

```

```
import { useState, useEffect } from 'react';

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

 const adjList: { [key: number]: { v: number; w: numbe
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
```

```

 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k: i, i: j, j: k, matrix: dist.map(r => [...r]), up: 1,
 log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
 activeCodeLine: 7,
 });
 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), up: 1,
 log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

<path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
</defs>
{Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={` ${u} - ${edge.v} `}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x} / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x} / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
});
}}}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a
 else if (isI) { fill = '#b45309'; stroke =
 else if (isJ) { fill = '#047857'; stroke =
 return (
 <g key={node.id} className="transition-

```

```

 {edge.v} ({edge.w})

)
 }}}
</div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Матрица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div className="flex justify-between items-center">
 <h4 className="text-lg font-bold text-rose-950">
 </div>
 <div className="overflow-x-auto">
 <table className="w-full text-center border">
 <thead><tr className="bg-rose-950/40 text-white">
 <th className="p-2 border-r border-rose-950">
 {nodeNames.map((n, idx) => <th key={idx}
 -400 font-bold' : ''}`}>{n.split(' ')[0]} {
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950">
 <td className={`p-2 bg-rose-950/40 text-white`}
 tStep.k === i ? 'text-amber-400' : ''}`}>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== i
 const isDest = currentStep.j === j});
 let bg = 'bg-slate-900/40 text-slate-500';
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50';
 else if (isVia) bg = 'bg-indigo-900/80 text-indigo-50';
 else if (isDest) bg = 'bg-rose-900/80 text-rose-50';
 <div className={`p-2 ${bg} text-white`}

```



```

import { Play, Pause, SkipForward, Undo, RefreshCw } from
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};
```

```

while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Д

 q], visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, de

 tack: [...q], visitedNodes: [...Array.from(

 }
 }
 }
}

steps.push({ type: 'backtrack', node: '', desc: `04

 om(vis)] });

return steps;
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("df
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? genera

 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [mode]);

 useEffect(() => {
 if (autoPlay) {

```

```

 { /* Edges */
 {edges.map((e, i) => {
 const u = nodes.find(n => n
 const v = nodes.find(n => n
 // Check if edge is part of
 const edgeTraversed = isVis

 return (
 <line key={i} x1={u.x}
 className={`stroke-
500' : 'stroke-emerald-500') : 'stroke-slate
);
 })}

 { /* Nodes */
 {nodes.map(n => {
 const visited = isVisited(n
 const current = isCurrent(n

 let fillColor = "fill-slate
 let strokeColor = "stroke-s
 let textColor = "fill-slate
 let radius = 18;

 if (visited) {
 fillColor = mode === 'd
 strokeColor = mode ===
 textColor = "fill-white
 }

 if (current && step.type !==
 strokeColor = "stroke-a
 textColor = "fill-amber
 radius = 22;
 }

 return (
 <g key={n.id} className=

```

```

 }}
 </div>
)))
 {!!step.queueOrStack || step.queue}
 <div className="text-slate-500" style={stepStyle}
 >
 </div>
 </div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={stepStyle} >
 {/* Controls */}
 <div className="flex bg-slate-900 border border-slate-800 rounded-lg p-2 text-slate-400 hover:text-white hover:bg-slate-800" style={stepStyle} >
 <Undo strokeWidth={3} size={16} />
 <button onClick={() => setAutoPlay(!autoPlay)}
 justify-center min-w-[80px] ${mode === 'default' ? '' : 'disabled'}
 >
 {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay); setStepIndex(steps.length - 1)}} className="p-2 text-slate-400 hover:bg-transparent" >
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay); setStepIndex(0)}}
 bg-slate-800 rounded-lg ml-2 border-l border-slate-700
 >
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border border-slate-800 p-2" style={stepStyle} >
 <div className="text-[10px] text-slate-400" style={stepStyle} >
 {steps.length}</div>
 </div>
</div>

```

```

 return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: 'Болят сердце',

```

```

 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выпис
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

 const preset = PATIENT_PRESETS[Math.floor(Math.rand
 const finalName = name || preset.name.split(' ')[0]
 const finalEmoji = preset.emoji;
 const finalSymptom = preset.symptom;

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 };

 const nextHeap = heapInsert(heap, newPatient).map(x
 // Mark specifically this new patient as 'in' in the
 const target = nextHeap.find(x => x.priority === pr
 const marked = nextHeap.map(x => x.id === (target?.

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.rand

```

```

 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat
 }));

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat
 addLog(` Успех! Пациент ${topPatient.name} п
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog(` База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
 x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }

```

```

<button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 opacity-40 disabled:cursor-not-allowed text-
 ve:scale-95 transition-all cursor-pointer f
>
 Привезти по Скорой (INSERT)
</button>

{/* Добавить кастомного */}
<form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border borde
 er-emerald-500 transition-colors placeholde
 />
 <div className="flex flex-col items-center min
 <span className="text-[9px] font-mono text-
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e
 className="w-20 accent-emerald-500 cursor
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap
 className="bg-teal-600 hover:bg-teal-500 di
 px-4 rounded-xl shadow-lg active:scale-95 t
 >

```



```

{heap.map(({patient, idx}) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'ani
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mono
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
}})

```

```

 }}
 </div>

 <div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500"
 </div>

 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-700 p-4"
 <div className="text-[10px] font-mono text-slate-400"
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? Действие :
 {entry}
 </div>
)}}
 </div>
 </div>
);
};

```

---

ФАЙЛ 41/82: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { createContext, useContext, useEffect, useState } from 'react';
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза — курсор на карточке" : "наведи"}
 </div>
 </div>
);
};
```

```
export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);
};
```

```

 step === 0
 ? "k ещё запрещена: путь $i \rightarrow j = 9$ "
 : step === 1
 ? "Разрешаем пересадку через k: $3 + 4 = 7$ "
 : " $7 < 9 \rightarrow D[i][j] = 7$ "
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```
export const ComponentsDemo: React.FC = () => {
```

```

 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"]],
 ["D", "E"]],
 ["F", "G"], ["G", "H"], ["F", "H"]],
];

```

```

const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],

```

```

 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
 {step >= 2 && (
 <text
 x={({a[0] + b[0]) / 2}
 y={({a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
)))
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}
 </Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0

```

-----  
 /\* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{  
 frames: { pos: Record<string, Pt>; edges: [string, st  
 captions: string[];  
 ms?: number;

 const step = useLoop(frames.length, ms);  
 const f = frames[step];  
 const color = (id: string) =>  
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id

return (  
 <Svg caption={captions[step]}>  
 {f.edges.map(([a, b], i) => (  
 <line  
 key={i}  
 x1={f.pos[a][0]}  
 y1={f.pos[a][1]}  
 x2={f.pos[b][0]}  
 y2={f.pos[b][1]}  
 stroke={C.edge}  
 strokeWidth={1.8}  
 style={{ transition: MOVE }}  
 />  
 ))}  
 {Object.entries(f.pos).map(([id, [x, y]]) => (  
 <g key={id} style={{ transform: `translate(\${x},  
 <circle r={13} fill={color(id)} stroke={C.idl  
 <text textAnchor="middle" dominantBaseline="c  
 {id}  
 </text>  
 </g>  
 ))}  
 </Svg>

);

ФАЙЛ 43/82: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "A"],
];

export const BfsDemo: React.FC = () => {
 const order = ["A", "B", "C", "D", "E", "F"];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order[step - 1] : null;
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`}
 {GRAPH_EDGES.map(([u, v], i) => (
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
))}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
 stroke={wave.includes(k) ? "#a5b4fc" : undefined} />
))}
 />
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
```

```

 {idx >= 0 && <circle cx={n.x + 30} cy={n.y
 {idx >= 0 && {
 <text x={n.x + 30} y={n.y - 12} textAnchor=
 x + 1}</text>
 }}
 </g>
);
 }}}
 <text x={224} y={62} textAnchor="middle" fontSize=
 <text x={224} y={74} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
 <Edge a={[60, 70]} b={[200, 70]} color={improved
 <text x={130} y={54} textAnchor="middle" fontSize=
 <Node x={60} y={70} r={20} fill={C.active} label=
 <Node x={200} y={70} r={20} fill={improved ? C.do
 <text x={60} y={106} textAnchor="middle" fontSize=
 <text x={200} y={106} textAnchor="middle" fontSiz
 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D

```



```

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 100],
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц`
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
 <text x={(pos[e.u][0] + pos[e.v][0]) / 2} y={pos[e.v][1]}
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

```

```

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45],
 };
 const edges: [string, string][] = [["A", "B"], ["B", "C"], ["C", "A"]];
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 && scc.includes(v);
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд

```

```

const step = useLoop(states.length, 800);
const items = states[step];
const popping = step >= 3;
return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮ таре."
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
 <text x={130} y={111 - i * 26} textAnchor="middle" />
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize={12} />
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к концу"
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
 <text x={53 + i * 54} y={71} textAnchor="middle" />
 </g>
))}
)
);
};

```

```

 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]]}
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
 </Svg>
);
};

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : null;
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, Math.min(levels.length, step))}`}
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]]}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)]}
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
 : "о префиксные суммы P"}>
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim}>
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i > l ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }}>
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (

```

```

 <text x={92} y={64} textAnchor="middle" dominantB
 <text x={206} y={34} textAnchor="middle" fontSize
 <circle cx={196} cy={84} r={13} fill={step === 1
 <text x={196} y={84} textAnchor="middle" dominant
 ?"}</text>
 <text x={130} y={124} textAnchor="middle" fontSize
 </Svg>
);
};

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.warn : C.idle}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle" font-size={10}>{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle" font-size={10}>{pi[idx]}</text>
 <rect x={32 + idx * 33} y={70} width={28} height={20}
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
))}
 <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
 <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" font-size={12}>
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

```

```

 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo
} from "./demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "./demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components"
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,

```

```

 vizId: string | null;
 onClose: () => void;
 }

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
 >

```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>(null);

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left, [anchor]});
 }, [anchor]);

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onKey, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onKey, true);
 };
 }, [anchor, onClose]);
};
```



```

 {entry.formal && {
 <p className="text-[11.5px] leading-relaxed" >
 {entry.formal}
 </p>
 }}

 {entry.complexity && {
 <div className="text-[11px] font-mono text-left" >
 Сложность: {entry.complexity}
 </div>
 }}

 {viz && simulatorId && {
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center gap-2
 rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 }}
 </div>
 </div>,
 document.body
);
};
};

```

ФАЙЛ 48/82: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {

```

```

 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

```

```

switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.
 Старый вес: <span className="text-r
 Вычисляем: $-2 + 0 - (-2) =$ <span cl
 </div>
);

```

```

 <marker id="arrow-slate" viewbox="0 0 10 5" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: slategray; stroke: none; stroke-width: 0px;"/>
 start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: none; stroke: slategray; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-slate-dim" viewbox="0 0 10 5" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: slategray; stroke: none; stroke-width: 0px;"/>
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: none; stroke: slategray; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-amber" viewbox="0 0 10 5" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: #FFD700; stroke: none; stroke-width: 0px;"/>
 start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: none; stroke: #FFD700; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-pink" viewbox="0 0 10 5" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: #FF69B4; stroke: none; stroke-width: 0px;"/>
 tart-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: none; stroke: #FF69B4; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-emerald" viewbox="0 0 10 5" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: #3CB371; stroke: none; stroke-width: 0px;"/>
 o-start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" style="position: absolute; top: 0; left: 0; width: 100%; height: 100%; fill: none; stroke: #3CB371; stroke-width: 2px;"/>
 </marker>
</defs>

```

```

{ /* Edges */
edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return {
 id: `e-${i}`,
 x: midX,
 y: midY,
 width: 100,
 height: 100,
 fill: edgeColorClass,
 stroke: edgeColorClass,
 stroke-width: 2,
 text: textHigh ? `H` : textEmer ? `E` : ``,
 text-x: textHigh ? 0 : textEmer ? 100 : 50,
 text-y: textHigh ? 50 : textEmer ? 50 : 50,
 text-align: textHigh ? 'center' : textEmer ? 'center' : 'center',
 text-size: 16,
 text-weight: 'bold',
 text-color: 'black',
 };
 }
});

```



```

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
}

```

```

 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый
 null,
 false,
 -1,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 initialEdges.forEach((e) => adj[e.u].push(e.v));

 // DFS 1: post-order list
 const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем`,
 u,
 false,
 -1,
);
 };
};

```

```

 dfs2(to, currentScc);
 }
}
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в`,
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
 null,
 true,
 sccId,
);

setSteps(generatedSteps);

```



```

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]"
 {/* Markers for directed arrows */}

```

```

const u = initialNodes.find((n) => n.id === edge.u)
const v = initialNodes.find((n) => n.id === edge.v)

// If transposed, draw reversed coordinates
const x1 = step.transposed ? v.x : u.x;
const y1 = step.transposed ? v.y : u.y;
const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 />

```

```

 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
 }}}
</svg>

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

```

```


 ЛОГ ОПЕРАЦИЙ:

<div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
 })}
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 № {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 px-2 py-1"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 px-2 py-1"
 >
 Вперед
 </button>
 </div>
</div>

```

```

 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)

```

```

 description: 'Оно соединяет две другие бесцветны
 type: 'info'
 }, ...prev]);
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный ц
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',

```

```

 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]);
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',

```

```

 type: 'warning'
 }, ...prev]));
 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
],
];

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {

```



```

 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в ку
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к верши
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {

```

```

() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет',
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. P
 type: 'success'
 }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',

```

```

 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]));
},
// Step 23: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));

```

```

// Component 2: {D, E, F, G, H, I} (colored gold/
setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
}));
setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red'
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue'
 }
 return e;
}));
setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.',
 type: 'success'
}, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge betw
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F,
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост
 description: 'Теперь Красный и Синий колхозы од
 -D с весом 5. Остальные мосты B-E(6) и C-F(9)
 type: 'warning'
 }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color:
 setEdges(prev => prev.map(e =>

```

```

 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 соединены ребрами. Нужно выбрать 8 из них, чтобы
 не было циклов и все вершины были соединены.',
 type: 'info',
 },
]);
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 тую вершину (Нар) и захватываем соседей!',
 type: 'info',
 },
]);
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 ет имеет своего соседа. Нужно выбрать 8 ребер,
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
 }
 };

```

```

const getColorClass = (color: string, id: number) => {
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-white';
 case 'blue': return 'bg-blue-500 border-blue-300 text-white';
 case 'purple': return 'bg-purple-600 border-purple-300 text-white';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIndex === 0) {
 return 'bg-slate-700 border-emerald-500 text-white';
 }
 }
};

```

```

<div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-400 p-2" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >

```



```

 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -
 }}
 </div>
 }}}
</div>

```

```

{stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
shadow-xl z-20">
 <p className={"font-bold text-base " +

```



```

<div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
)
 }
 >
 <div className="flex items-center gap-2"
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCirc
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)})}
</div>
</div>
</div>
</div>

```

```

{/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-40
 <h4 className="text-xl font-bold text-white
 </div>
 </div>
</div>

```

```

 </p>
 <p className="text-2xl font-black text-slate-900">
 <p className="text-slate-400 text-xs mt-2">
 </div>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
ождении:</p>
 <p className="text-xs text-slate-300 leading-5">
ди и красим вершины в один цвет, бракуя циклы.
 </div>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-2xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-indigo-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </pre>

```

```
def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
```

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'boruvka' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-v

<p className="text-slate-400 text-xs mt

p>

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-ro

</p>

<p className="text-2xl font-black text-v

<p className="text-slate-400 text-xs mt

```
 });
 };
```

---

ФАЙЛ 51/82: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: ' $O((V+E) \log V)$ ',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: ' $O(V \cdot E)$ ',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
```

---

ФАЙЛ 52/82: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);

 useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
```

```
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
)}
</>
);
};
```

---

ФАЙЛ 53/82: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Code } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
 const [busy, setBusy] = useState(false);

 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };
};
```

```

onClick={() => setActiveTab('simulator')}
className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
}`}
>
 <Cpu className="w-4 h-4" /> Тренажёры
</button>
<a
 id="download-pdf-btn"
 href="/export/full_code_all_pages.pdf"
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
title="Скачать полный PDF сборник всех стра
>
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href="/export/full_code_all_pages.txt"
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
title="Скачать полный TXT файл со всем кодо
>
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}

```

```

const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
function findPt(id:number) { return pts.f
function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
}
const crosses: number[] = [];
for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x
 stroke={xing ? "#f43f5e" : "#4f46e5"}
 }
const labels = ["A","B","C","D","E"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill=
 <text x={p.x} y={p.y+3} textAnchor="m
 </g>);

```



```

 if (intersect(edges[i][0],edges[i][1],e)
 if (!crosses.includes(i)) crosses.push(i)
 if (!crosses.includes(j)) crosses.push(j)
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#f43f5e" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" style={{z-index: 20}}>
 $V=6, E=9 \Rightarrow 2V-4 = 8$ (двудольный)
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950/40 padding-10">
 ▲ Запомни: K_5 и $K_{3,3}$ – два кита непланарности.
 Гомеоморфный K_5 или $K_{3,3}$ – он непланарен. Теорема Кюри.
</div>
</div>
);
}

```

```

 { id: 'q2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'red',
 text: 'Пожалуйста, принеси мне супчик! Я голодный! Я хочу
ишу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐱', color: 'blue',
 text: 'Пожалуйста, принеси мне сметаны, плиз.', animState: 'idle' },
]);

```

```

const [servedHistory, setServedHistory] = useState<string[]>([]);
const [isServing, setIsServing] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

```

```

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

```

```

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяжело!');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
}, [isServing]);

```

```

const preset = PRESETS[counter % PRESETS.length];
counter++;

```

```

const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
};

```

```

setQueue(prev => [...prev, newGuest]);
addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

```

```

setTimeout(() => {
 setQueue(prev => prev.map(c => c.id !== newGuest.id ? c : newGuest));
}, 450);

```

```

 т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻',
 апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'red',
 без сметаны, плиз.', animState: 'in' },
]));
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState
 }, 500));
};

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест.
 Новые гости встают строго в конец очереди.
 Здесь нет обхода и блата — только строгий порядок.
 </p>
 </div>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600
 text-white font-medium rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-indigo-500
 disabled:opacity-40 disabled:cursor-not-allowed transition-all cursor-pointer">
 >

```

```

 <div className="absolute -top-10 left-10" >
 <div className="w-1.5 h-6 bg-slate-400" >
 <div className="w-2 h-4 bg-slate-400" >
 </div>

 </div>
 <div className="text-center">

 ПОВАР

 </div>
 </div>

 { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
 <div className="flex flex-col items-center" >

 </div>

 { /* СПИСОК ОЧЕРЕДИ */ }
 <div className="flex-1 flex items-end gap-2" >
 {queue.length === 0 ? (
 <div className="flex-1 flex flex-col" >
 </
 <p className="text-xs font-bold font-mono" >
 <p className="text-[10px]" >Все сыты
 </div>
) : (
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length - 1;
 return (
 <div
 key={customer.id}
 className={`flex flex-col items-center gap-2`}
 >

```

```

 })
 })
 </div>

 </div>
</div>

 { /* Пол кухни */ }
 <div className="w-full h-3 bg-gradient-to-r from-blue-500 to-purple-500">
</div>

 { /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
 <div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[11px] font-mono">
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-end gap-2 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse text-white">
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-between p-4">
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2">
 0
 <p className="text-xs font-bold font-mono">Здесь нет сытых гостей</p>
 <p className="text-[10px] mt-1">Здесь 6 сытых гостей</p>
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-full">
 {servedHistory.map((item, idx) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded-2xl border border-slate-700 flex items-center gap-2 text-[11px] text-white">

```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!'
);
 const [isTalking, setIsTalking] = useState<boolean>(f
 const [isBlinking, setIsBlinking] = useState<boolean>

 // Регулярное моргание рыбы
 useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
 }, []);

 // Эффект разговора при смене текста
```

```

 });

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : leftX + paddingX / 2;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для заданного списка S, чтобы расставить fail-ссылки!`
);
 };
};

```

```

 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на гл
 очке суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полны
 ь полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.ch

 for (const [char, u] of Object.entries(rNode.children)
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char]
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char]
 updatedNodes[u].fail = failTarget;

```





```

tween min-h-[140px]">
{ /* Треугольник баббл */
<div className="absolute -left-3 top-1/2 -tran
 slate-800 border-b-[12px] border-b-transpar

<div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
</div>

<p className="text-slate-200 text-base leading
 {speechText}
</p>

<div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex
 <Info className="w-3.5 h-3.5 text-blue-40
 Статус: {simulationStep === 0 ? 'Бор гото
 оен!'}
</div>

<div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-em
 -1 shadow-lg shadow-emerald-900/40 transition
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-5
 dow-lg shadow-blue-900/40 transition-transf
 >
 Шаг BFS
)}

```

```

 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 { /* Draw Edges */ }
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 })}
 </pre>

```

```

 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
)}
 </g>
);
 }
}

```

```

 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
 </form>
 </div>

 {/* Таблица Вершин и суффиксных ссылок */}
 <div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>
 <tr className="border-b border-slate-700" >
 <th className="py-2 px-3" >ID Вершины</th>
 <th className="py-2 px-3" >Символ</th>
 <th className="py-2 px-3" >Родитель</th>
 <th className="py-2 px-3" >fail-ссылка</th>
 </tr>
 </thead>
 <tbody>
 {nodes.map((node) => (
 <tr>
 <td>{node.id}</td>
 <td>{node.symbol}</td>
 <td>{node.parent}</td>
 <td>{node.fail}</td>
 </tr>
))}
 </tbody>
 </table>
 </div>
 </div>
 </div>
)

```

```

 {node.char}

 </td>
 <td className="py-2.5 px-3 text-slate-500"
 {node.id === 0 ? '-' : `#${node.id}`}>
 </td>
 <td className="py-2.5 px-3"
 {node.id === 0 ? (

) : (
 <span className="bg-indigo-950 text-white"
 #{node.fail}>

)}>
 </td>
 <td className="py-2.5 px-3"
 {node.id === 0 || node.term_link ? (
 <span className="text-slate-500"
) : (
 <span className="bg-rose-950 text-white"
 0 transition-colors cursor-help" title={`Click to expand`}>
 #{node.term_link}>

)}>
 </td>
 <td className="py-2.5 px-3 text-slate-500"
 {Object.keys(node.children).length === 0 ? '0' : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}>
 </td>
 <td className="py-2.5 px-3"
 {node.words.length === 0 ? (

) : (
 <span className="bg-emerald-900 text-white"
 {node.words.join(', ')}>

```

```

const n = arr.length;
const tree: Record<number, number> = {};
function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
}
go(1, 0, n - 1);
return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

```

```

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];

```

```

};

```

```

void CODE;

```

```

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})` });
 }
}

```

```

] = ${tree[v]}`, codeLine: 4, treeState: tree,
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v],
 const left = go(2 * v, l, m, qL, qR);
 const right = go(2 * v + 1, m + 1, r, qL, qR);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 rgeChildren: [2 * v, 2 * v + 1], arrayHighlight: [v],
 return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]:`,
 ghlightNodes: [1], arrayHighlight: [qL, qR], result: ans,
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

 steps.push({ id: id++, desc: `Capt: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`,
 codeLine: 3, treeState: tree, highlightNodes: [l, r],
 while (l < r) {
 const highlights: number[] = [];

```



```

 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"

```

```

];
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"

```

```

];
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"

```

```

];

```

// ===== PLAYER COMPONENT =====

```

function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);

```

```

let cls = 'bg-slate-800 border-slate-700 text-slate
if (isHigh) cls = `${clr.ring} text-white ring-2 sc
else if (isChild) cls = `${clr.childRing} text-ambe
else if (has) cls = `${clr.filled} text-slate-200`;

return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
 ${cls}`>
 <span className="text-[7px] opacity-60 font-m
 [{nd.l
 <span className="text-xs font-extrabold font-i
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full l
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
 </div>
)}
 </div>
);
}, [step, clr]));

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 /* Header */
 <div className={`px-4 py-2.5 flex items-center ju
 -emerald-950/50' : 'bg-amber-950/50'} border-b l
 <h4 className="font-bold text-sm text-white fle
 <span className="text-[10px] font-mono text-sla
 </div>

 /* Array strip */

```

```


{step.desc}
{step.result !== undefined && (

 {step.result}

)}
</div>

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t border-slate-800" >
 <div className="flex items-center bg-slate-900 rounded" >
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
 <RotateCcw className="w-4 h-4 text-emerald-300" />
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(700); }}>
 <RotateCcw className="w-4 h-4 text-emerald-300" />
 </button>
 <button onClick={() => setPlaying(!playing)}>
 <Play className="w-4 h-4 text-emerald-300" />
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(120); }}>
 <RotateCcw className="w-4 h-4 text-emerald-300" />
 </button>
 </div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))}>
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
</div>

{/* Progress bar */}
<div className="h-1 bg-slate-950"><div className="bg-emerald-500" style={
 {width: ((currentStep + 1) / total) * 100}%></div>
</div>

```

```

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
 {/* ---- TOP BAR: array editor + query range ---- */}
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">
 Array input
 </label>
 <button type="button" onClick={randomize} className="text-white">
 Refresh
 </button>
 </div>
 <div className="flex gap-2">
 <input
 type="text"
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2">
 Применить
 </button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

 {v}

))}

 A[0..arr.length-1]

 </div>
 </form>

 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-indigo-500">

```

```

 -slate-200'}`}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} class
 on-colors ${view === 'queries' ? 'border-emera
 er:text-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} class
 n-colors ${view === 'theory' ? 'border-blue-5
 te-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Спра
 </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху
 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}

```



```

const filtered = q ? chapters.filter((c) => c.title
const map = new Map<string, typeof chapters>();
for (const c of filtered) {
 const key = c.category?.trim() || "Прочие темы";
 if (!map.has(key)) map.set(key, []);
 (map.get(key) as typeof chapters).push(c);
}
return Array.from(map.entries());
}, [query]);

```

```

return (
 <aside className="w-full bg-slate-900/80 lg:bg-tran
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400"
 <Compass className="w-4 h-4 text-indigo-400"
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 abs
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border bo
 500 focus:outline-none focus:border-indigo-!
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
 </div>

```

```

<div className="flex-1 overflow-y-auto px-3 pb-4

```

```
 </div>
 </div>
)))
 </div>

 <div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-300 to-indigo-400
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex items-center">
 <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по наведению.
 </p>
 <p className="text-slate-400 italic">Стрелки ← и → переключают вид.
 </div>
 </aside>
);
};
```

---

ФАЙЛ 59/82: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';
```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
}
```



```
// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },
 { name: 'Кастрюля от супа', icon: ' ' },
 { name: 'Чашка от кофе', icon: '=' },
 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты.");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};
```

```

const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(`  Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(`  Выбран самый "тяжелый" кандидат:`);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"', probability: 0.1 },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"', probability: 0.2 },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"', probability: 0.3 }
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

```

```

<button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 text-white'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-300'}
>
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-300'} opacity-80`} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Queue</div>
 <div className="text-xs opacity-80">FIFO</div>
 </div>
 </div>
 BFS
</button>

<button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 text-white'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-300'}
>
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-300'} opacity-80`} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Heap</div>
 <div className="text-xs opacity-80">Приоритетная</div>
 </div>
 </div>
 Beam Search
</button>

```

```

 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {popMessage && {
 <div className="bg-blue-950/60 border border-
imate-fade-in">
 <span className="inline-block w-2 h-2 round
 {popMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стор
 <p className="text-xs mt-1">Добавьте гр
 </div>
 } : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map(({plate, in
 const isTop = index === 0;
 return {
 <div
 key={plate.id}
 className={`w-full flex items-cen
 isTop

```

```

 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
 </div>
 <div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

{dequeueMessage && (
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
)}

```

```


 })
 {
 <span className={`font-bold text-
 {person.name}

 <span className={`text-[10px] font
 isFirst ? 'bg-indigo-500/30 tex
 }`}>
 {isFirst ? 'Получает сын' : `В

 </div>
 });
 }}}

 <div className="flex flex-col items-cen
 -600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
</div>
})
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
}}
```

```

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
```

```

 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 text-sm font-medium text-white bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed rounded-md p-2"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
 </form>

 {heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/40 p-3 animate-fade-in">

 {heapMessage}
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2xl">

```

```

 <span className="text-[10px
-0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд

 }}
</div>
</div>

<div className="flex items-center
 {/* Custom progress bar */}
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
</div>
);
 }}}}
</div>
)}
<div className="mt-6 text-xs text-slate-500
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};
```



```

 (i.entry.hint ?? "").toLowerCase().includes(q)
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
}, [query]));

return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabold">
 <p className="text-sm text-slate-400 leading-relaxed">
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2"
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
 </div>

 {items.length === 0 && <p className="text-slate-500">
 <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3">

```

ФАЙЛ 61/82: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево,
 * поднять отмеченный узел в корень. Клик по узлу = один поворот.
 * Порядок поворотов выбираешь сам; правильный ли он пока неясно,
 * только после того, как узел окажется наверху (или по центру).
 */
```

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просечка (zig-zag-zig) */
export const PRESETS: { name: string; hintCase: string; target: number; build: () => TNode } = [
```

```
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(7) };
 return { key: 6, left: n4, right: leaf(7) };
 }
 },
```

```

 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

```

```

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

```

```

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

```

```

 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 });
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.build());
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; color: string }>([]);
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() => preset.build(), [preset]);
 const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
 const expected = useMemo(() => (solved ? null : canonicalPath(target)), [target, solved]);

 const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 }
}

```

```

<div className="flex gap-2 mb-3 overflow-x-auto" style={{ width: '100%' }}
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11pt]
 ${i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-gray-800"}
 `}
 >
 {p.name}
 </button>
))}
</div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-800" style={{ width: '100%' }}
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
 />
);
 })}
 </svg>

```

```

 -white disabled:opacity-40 text-xs font-bol
 >
 <Undo2 className="w-3.5 h-3.5" /> Отменить
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-3
 text-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-3
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-200"
 }`
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 {showAnswer ? "Скрыть разбор" : "Сдаться / посмотреть"}
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

</div>

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

{solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 >

```

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
```

```
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
```

```
 *
```

```
 * Что сделано ради понятности:
```

```
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
```

```
 * * сразу видно, что дерево остаётся деревом поиска;
```

```
 * * под деревом печатается обход слева направо: он ОДНОшаговый;
```

```
 * * это и есть доказательство, что поворот ничего не ломает;
```

```
 * * кадр «прицел» ничего не двигает, только подсвечивает;
```

```
 * * на кадре «результат» остаются призраки – пунктирные узлы
```

```
 * * местах и стрелки «откуда → куда»;
```

```
 * * по умолчанию анимация НЕ крутится сама: пользователь
```

```
 * * идёт в своём темпе. Автопрокрутка включается кнопкой
```

```
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
```

```
 pos: Pos;
```

```
 edges: [NodeId, NodeId][];
```

```
 kind: "start" | "aim" | "result";
```

```
 title: string;
```

```
 text: string;
```

```
 /** Пара, которую крутим на этом шаге: кадр-«прицел».
```

```
 focus?: [NodeId, NodeId];
```

```
 /** Кто переехал на этом кадре – рисуем призрак и стрелки.
```

```
 moved?: NodeId[];
```

```
 ms: number;
```

```
}
```

```
interface Case {
```

```
 key: "zig" | "zig-zig" | "zig-zag";
```

```
 label: string;
```

```

 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 – 20",
 text: "Ничего пока не двигается. Смотри на подсве
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддерево
 ева от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или об
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },

```



```

 edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 в корне, бамбук стал кустом",
text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
moved: [{"x", "p", "A", "B"}],
ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p – дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым – нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,

```

```

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

const COLOR: Record<string, { fill: string; stroke: string }> = {
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id === "B";

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C }> = ({
 frame,
 prev,
 keys,
}) => {
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
 const ghosts = frame.kind === "result" ? (moved ?? []) : [];

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" refX="0" refY="0">
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 })}
 </svg>
);
};

```

```

 }}}

 {(Object.keys(pos) as NodeId[]).map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id))
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size={10}
 {id}
 </text>
)}
 </g>
);
 })}
 </svg>

```

```

 {c.label}
 </button>
)))
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корр.
 ({active.rotat
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала
 <div className="mb-3 rounded-xl border border-slate-200">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`>
 >
 {current.kind === "aim" ? "прицел" : current.title}

 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading-5">
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

 {/* обход слева направо – одинаковый на всех ка
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход

```

```

>
 ← Назад
</button>
<button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
>
 {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
>
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1x" : "1x"}
</button>
<button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-1

```

```

 left: SplayNode | null;
 right: SplayNode | null;
 x?: number;
 y?: number;
}

// Simple BST insertion to build initial tree
const insertBST = (root: SplayNode | null, key: number)
 if (!root) return { key, left: null, right: null };
 if (key < root.key) {
 root.left = insertBST(root.left, key);
 } else if (key > root.key) {
 root.right = insertBST(root.right, key);
 }
 return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
 return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode |
 if (!node) return null;

```

```

let path: { node: SplayNode; dir: "left" | "right" }
let current: SplayNode | null = root;

// Find the element or the element where search failed
let lastNode: SplayNode = root;
while (current) {
 lastNode = current;
 if (key === current.key) {
 path.push({ node: current, dir: null });
 break;
 } else if (key < current.key) {
 path.push({ node: current, dir: "left" });
 current = current.left;
 } else {
 path.push({ node: current, dir: "right" });
 current = current.right;
 }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в дереве`);
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. Инициализируем новый узел`
);
} else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем`
);
}

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {

```

```

 node.right.right = splayAction(node.right.right, target);
 node = leftRotate(node);
 steps.push(
 `Вращение Zag-Zag (Левый поворот родителя д...
);
 }

 if (!node.right) return node;
 steps.push(
 `Финальный поворот Zag (Левое вращение корня
);
 return leftRotate(node);
}
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree,

```



```

 });

 const drawTree = cloneTree(tree);
 computeCoordinates(drawTree, 300, 40, 140);

 // Render lines and nodes
 const renderLines = (node: SplayNode | null): React.R
 if (!node) return [];
 let lines: React.ReactNode[] = [];
 if (node.left && node.left.x && node.left.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.left.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`r-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;

```

```

 circles = circles.concat(renderNodes(node.right));
 return circles;
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 { /* Header */ }
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <BookOpen className="w-5 h-5 text-indigo-400"
 Интерактивное Splay-дерево
 </h3>
 <p className="text-xs text-slate-400">
 Кликните на вершины дерева, чтобы «вытащить»
 Splay.
 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12
 { /* Playfield */ }
 <div className="lg:col-span-7 bg-[#0f172a] p-4
 <svg
 className="w-full h-full max-w-[600px] max-
 viewBox="0 0 600 320"
 >
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
 </svg>

 { /* Quick interactive controls */ }
 <div className="absolute bottom-4 left-4 right
 <form onSubmit={handleFind} className="flex
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target
 className="w-24 bg-slate-950 text-white

```

```
</div>
```

```
{/* Logs */}
```

```
<div className="lg:col-span-5 bg-slate-950 border-1
to overflow-y-auto custom-scrollbar">
```

```
<h4 className="text-xs font-bold text-slate-400">
```

```
Ход вращения (Splay-логи)
```

```
</h4>
```

```
<div className="flex-1 space-y-2 mb-4 overflow-y-auto">
```

```
{log.map((step, idx) => {
```

```
<div
```

```
key={idx}
```

```
className={`text-xs p-2.5 rounded transition-colors`}
```

```
idx === log.length - 1
```

```
? "border-l-2 border-indigo-500 bg-slate-900/50"
```

```
: "text-slate-400 bg-slate-900/50"
```

```
}`}
```

```
>
```

```
{step}
```

```
</div>
```

```
}}}
```

```
</div>
```

```
<div className="border-t border-slate-800 pt-4">
```

```
<p>
```

```

```

```
у самого корня. Делаем 1 вращение.
```

```
</p>
```

```
<p>
```

```

```

```
оба левые или оба правые. Вращаем деда, по
```

```
</p>
```

```
<p>
```

```

```

```
колесо. Вращаем узел в разные стороны.
```

```
</p>
```

```
</div>
```

```
</div>
```

## 2. Эффект качелей (Swing)

```
</p>
<p className="text-slate-400 leading-relaxed">
 Если агент запрашивает по очереди{" "}
 <code className="text-rose-400">[10]</code>
 <code className="text-rose-400">[60]</code>
 углы), дерево будет превращаться в палки
 сторону. Первая операция Splay(60) стоит
 <strong className="text-white">O(n)
 глубину n. Вторая Splay(10) заставляет
 Постоянный свинг убивает производительность
 сложности <strong className="text-white">O(n)
</p>
</div>
<div className="mt-3 text-[10px] text-rose-400">
 Кэшировать свингующие пары на концах — фа
</div>
</div>
```

```
<div className="bg-slate-900/60 p-4 rounded-lg">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 3. Амортизация $O(\log n)$
 </p>
 <p className="text-slate-400 leading-relaxed">
 Парадокс: один запрос за{" "}
 <code className="text-emerald-400">O(n)</code>
 всю структуру вдвое более плоской за счет
 при двойных вращениях {
 <code className="text-emerald-400 font-mono">O(n)</code>
 }. Дорогая операция инвестирует в дешевые
 операций. Поэтому амортизированное время
 <strong className="text-white">O(log n)
 </p>
 </div>
</div>
<div className="mt-3 text-[10px] text-emerald-400">
 Каждое "растяжение" компенсируется "сжатием"
</div>
```

```

 { id: 'p1', name: 'Тарелка из-под спагетти', emoji:
 ate: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji:
 State: 'idle' },
 { id: 'p3', name: 'Блюдец от торта', emoji: '
 tate: 'idle' },
]);

```

```

const [cleanRack, setCleanRack] = useState<Plate[]>([
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([

```

```

const addLog = (msg: string) => setLog(prev => [msg,

```

// PUSH: Положить грязную тарелку сверху стопки

```

const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
}

```

```

const preset = PLATE_PRESETS[counter % PLATE_PRESET
counter++;

```

```

const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
};

```

```

setDirtyStack(prev => [...prev, newPlate]);
addLog(` PUSH: Положили ${newPlate.emoji} сверху с

```



```

 { /* Визуализация раковины */ }
 <div className="absolute top-4 right-4 flex items-center justify-center" style={{
 background-color: 'slate-800/80', text: '[10px] font-mono' }>
 <span className="w-1.5 h-1.5 rounded-full border border-white" style={{
 background-color: 'white', border: '2px solid white' }} />
 РАКОВИНА
 </div>

 { /* Мыльные пузыри при мытье */ }
 { showSoap && (
 <div className="absolute inset-0 pointer-events-none" style={{
 background-color: 'white', opacity: 0.5, width: '100%', height: '100%' }}>
 <div className="absolute w-6 h-6 rounded-full border border-white" style={{
 background-color: 'white', border: '2px solid white' }} />
 <div className="absolute w-4 h-4 rounded-full border border-white" style={{
 background-color: 'white', border: '2px solid white' }} />
 <div className="absolute w-5 h-5 rounded-full border border-white" style={{
 background-color: 'white', border: '2px solid white' }} />
 <div className="absolute text-4xl anim-curve" style={{
 color: 'white', font-family: 'monospace' }}>
 </div>
 </div>
) }

 { dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center" style={{
 background-color: 'slate-800', color: 'white' }}>
 <span className="text-6xl mb-3 animate-pulse" style={{
 color: 'white', font-family: 'monospace' }}>
 <p className="text-white font-black text-2xl" style={{
 font-family: 'monospace' }}>
 <p className="text-slate-500 text-xs mt-1" style={{
 font-family: 'monospace' }}>
 </p>
 </div>
) : (
 <div className={`flex-1 flex flex-col justify-center items-center`} style={{
 background-color: 'slate-800', color: 'white' }}>
 { /* Указатель TOP */ }
 <div
 className="absolute right-8 flex items-center justify-center"
 style={{ bottom: `${24 + topIndex * 44}px` }}>
 <span className="text-xs font-mono text-white" style={{
 font-family: 'monospace' }}>
 <ArrowUp className="w-3.5 h-3.5" />

```

```

 /* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */
 <div className="md:col-span-5 bg-slate-900 rounded-2xl p-4" >
 <div className="absolute top-4 left-4 text-[10px] font-mono" >
 Сушилка для чистой посуды
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-2" >
 <div className="order-emerald-800/80 text-[9px] font-mono" >
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
 </div>

 <div className="flex-1 flex flex-col justify-center gap-2" >
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2" >
 0
 <p className="text-xs font-bold font-mono" > Нет тарелок
 <p className="text-[10px] mt-1" >Здесь будут тарелки
 </div>
) : (
 <div className="flex flex-col gap-2 w-full" >
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full border border-slate-700" />
 <div className="flex items-center justify-center gap-2 opacity-70" >
 {plate.name}
 {plate.status}
 </div>
 </div>
 </div>
)}
 </div>
 </div>

 /* Столешница сушилки */
 <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" />
 </div>

```



```

 let j = pi[i - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i,
 ;

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 яд назад: откат j = pi[${j - 1}] = ${pi[j -
 j = pi[j - 1];
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записыва

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;

```

```

 }
 return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
 const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

```

```

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

```

```

 const { s, steps, patternLength } = useMemo(() => {
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);

```

```

 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
 const timer = useRef<NodeJS.Timeout | null>(null);

```

```

 // Reset when inputs or mode change
 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [s, mode]);

```

```

 // Timer loop for autoPlay
 useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { setAutoPlay(false);
 return prev + 1;
 }
 });
 }, 1200);
 }
 }, [steps]);

```

```
</div>
```

```
<div className="bg-slate-950 rounded-xl border-2 border-slate-900 px] flex items-center justify-start">
```

```
 <div className="flex gap-1.5 px-4 h-full">
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length
 const isSeparator = index === pattern.length
```

```

 let borderColor = "border-slate-900"
 let bgColor = "bg-slate-900"
 let textColor = isSeparator ? "text-slate-400" : "text-slate-900"
```

```
 // KMP Logic
```

```
 if (mode === "kmp") {
 if (step.highlight?.includePattern) {
 borderColor = "border-emerald-500"
 bgColor = step.match === true ? "bg-emerald-500" : "bg-slate-900"
```

```
 } else {
 borderColor = "border-slate-900/40"
 }
 }
 })
}
```

```
 // Z Logic
```

```
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900"
 borderColor = "border-emerald-900"
 }
 }
```

```
 if (step.zTarget === index) {
 borderColor = "border-emerald-900"
 bgColor = step.match === true ? "bg-emerald-900" : "bg-slate-900"
```

```
 } else {
 borderColor = "border-slate-900/40"
 }
 })
}
```

```
 return (
```

```
 <div key={index} className={className} style={style}>
 {/* L/R markers for Z * */
```

```

 </div>
 }}

 {/* Z fingers */}
 {mode == "z" && step.i
 <div className="abs
e z-20">
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}
 {mode == "z" && step.z
 <div className="abs
 <span className
 </div>
 }}

 {/* Found flash effect
 {step.found && mode ==
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 {step.found && mode ==
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 </div>
)
 }}}
 </div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}

```

Мы пишем **<b>Шаблон + # + Текст</b>**.

Когда значени {mode === 'kmp' ? "пр  
о было найдено!

Пройди шаги вручную, посмотри, как  
ля пропуска работы (в Z).

</p>

</div>

{/\* Code Sneak Peek \*/}

<div className="bg-slate-900 border border-

<div className="bg-slate-800 px-4 py-2 b

late-400 uppercase">

<span>Код C++ ({mode === 'kmp' ? 'КМП

</div>

<pre className="p-4 text-xs font-mono te

{mode === 'kmp' ? `vector<int> pi(s.length());

for (int i = 1; i < s.length(); i++) {

int j = pi[i-1];

while (j > 0 && s[i] != s[j]) j = pi[j-1];

if (s[i] == s[j]) j++;

pi[i] = j;

}` : `int l = 0, r = 0;

for (int i = 1; i < n; i++) {

if (i <= r) z[i] = min(r - i + 1, z[i - l]);

while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;

if (i + z[i] - 1 > r) {

l = i;

r = i + z[i] - 1;

}

}`}

</pre>

</div>

</div>

);

}

---

```
 { u: 3, v: 4 },
];
```

```
interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];
```

```
 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };
 });
```

```
 recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
```

```

 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
 }

 recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
 setSteps(listSteps);
 setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

```

```

 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4F81BD" />
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#808080" />
 </marker>
 </defs>

 {/* Edges */}
 {dagEdges.map((edge, idx) => {

```



```

 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"

```

```

 })
 </div>
</div>

{/* Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx === steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Далее
 </button>
 </div>
</div>

```

```
export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>

interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
```

```

 trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}

```

```

export interface MergeResult {
 root: TNode | null;
 trace: { atId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; }[];
}

```

```

export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null => {
 if (!a || !b) {
 trace.push({ atId: a?.id ?? null, bId: b?.id ?? null, note: `Объединение целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ atId: a.id, bId: b.id, chosenId: a.id, note: `Выбор ${a.key} с его правое поддерево с (${b.key}, ${b.grade})` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ atId: a.id, bId: b.id, chosenId: b.id, note: `Выбор ${b.key} с его левым поддерево с (${a.key}, ${a.grade})` });
 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

```

```

export function eraseNode(root: TNode | null, key: number): MergeResult {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {

```

```

 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

/\* ----- раскладка сцены дерева -----

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 44 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) es.push({ from: n.id, to: n.left.id });
 if (n.right) es.push({ from: n.id, to: n.right.id });
 walk(n.left);
 walk(n.right);
 };
 walk(root);
 return es;
};

```

```

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
}

```

```

 <g key={`tn-${n.id}`} transform={`translate
 <circle r={pulse ? 24 : 20} fill={nodeColor}
 } strokeWidth={pulse ? 4 : 2} className="tr
 <text y={-2} fill="#fff" fontSize="13" fo
 <text y={13} fill="#cbd5e1" fontSize="10"
 </g>
);
 }}}
</svg>
 {subtitle && <p className="text-xs text-slate-400
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => vo
}

```

```

 {interactive && selected && cursor && {
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDash=[5,5]
 />
 }}

 {points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key === p.key || e.b.key === p.key))
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ? () => {setSelected(p)} : undefined}
 onPointerUp={interactive && onPointUp ? () => {setSelected(null)} : undefined}
 style={interactive ? { cursor: "pointer" } : undefined}
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8" : "#ccc"}
 strokeDash={isSel ? [5,5] : done ? [1,3] : [1,0]}
 />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} font-family="monospace">{p.key}</text>
 </g>
);
 })}

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.grade)} r={10}
 stroke="#ccc" strokeDash=[1,0]
 />
 <text x={PX(pending.key)} y={PY(pending.grade) + 3.5} font-family="monospace">{pending.key}</text>
 </g>
)}
 </svg>
);
};

```

```

 b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
 edges: GameEdge[],
 x: Pair,
 y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
 if (x.key === y.key && x.grade === y.grade) return { ok: true };
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 (pid(e.a) === pid(y) && pid(e.b) === pid(x))) {
 return { ok: false, reason: "такая линия уже проведена" };
 }
 }
 if (x.grade === y.grade) return { ok: false, reason: "не может быть" };
 const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];
 if (edges.some((e) => pid(e.b) === pid(child))) {
 return { ok: false, reason: `у точки ${child.key} уже есть родитель` };
 }
 // цикл: родитель уже сидит в компоненте ребёнка (ребёнок уже в компоненте родителя)
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a))) {
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 }
 if (childComp.has(pid(parent))) return { ok: false, reason: "уже есть родитель" };
 return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap всегда идёт от старшего к младшему"
 };
}

```



```

 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v.key);
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) {
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяет
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
 };
 walk(roots[0]);
 if (seen.size !== n) bstOk = false;
 for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
 }
 return { heapOk, singleParent, connected, countOk, bstOk };
}

/** Дерево из проведённых линий – для показа результата

```

```

const [playing, setPlaying] = useState(false);

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left, right: node.right }, b: { key: node.left.key, group: node.left.group, left: node.left.left, right: node.left.right } });
 if (node.right) { es.push({ a: { key: node.key, group: node.group, left: node.left, right: node.right }, b: { key: node.right.key, group: node.right.group, left: node.right.left, right: node.right.right } });
 walk(node.left);
 walk(node.right);
 };
 walk(userTree);
 return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges], [canonEdges]);

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.length : canonEdges.length / 2;
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, canonEdges]);

```

```

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};

const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) — теперь`);
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
};

const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};

const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100,
 y: ((e.clientY - rect.top) / rect.height) * 100 });
};

```

```

 кто раньше в списке (сортировка ста
 {parsed.equalX && <div>✂ Есть равные x
 de.key → left).</div>}
 </div>
 })
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4" >
 <div className="flex flex-wrap items-center justify-center" >
 <h4 className="text-base font-bold text-emerald-400" >
 <div className="flex items-center gap-2" >
 {phase === "sort" ? (
 <>

 {["quicksort", "counting"] as const }.map((a) =>
 <button
 key={a}
 onClick={() => { setAlgo(a); setSort(a); }}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-white
 600 text-white" : "bg-slate-700 text-slate-400`}
 >
 {a === "quicksort" ? "quicksort (ср
 </button>
)
) : (
 <>

 <button
 onClick={() => startConnect("y-desc")}
 className={`px-3 py-1.5 rounded-lg text-white
 bg-emerald-600 text-white" : "bg-slate-700
 >
 по y ↓ (алгоритм)
 </button>
 <button

```

```

 {comparisons} ($n \cdot \log_2 n$).</>
) : (
 <> counting: точки в корзины по значен
 , потому что y – маленькие целые.</>
)
) : (
 <>Отсортировано по y . Смета сортировки: q
 0 ($n+k$ операций). Дерево в обоих слу
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказан
 ево: линии вниз по y , обход по x отсортиров
) : gameMsg ? (
 <>▲ {gameMsg}</>
) : (
 <>Клики точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
)}
</div>
</div>

```

```

{ /* Чек-лист инвариантов */ }
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`} >
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} y
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
)}


```

```

 </div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), []);
 const split = useMemo(() => splitTree(buildTree(POINTS), stepIdx), [stepIdx]);
 const steps = split.trace;
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const inL = new Set<NodeId>();
 const inR = new Set<NodeId>();
 steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

 const lTree = split.l;
 const rTree = split.r;

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Number(e.target.value)); }}>
 {
 [2, 3, 6, 7, 9, 12, 45].map((k) => <option key={k} value={k}>{k}
 }
 </select>
 <button onClick={() => setStepIdx(Math.max(0, stepIdx - 1))} className="bg-slate-700 text-white disabled:opacity-30">←
 <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))} className="font-bold bg-indigo-600 text-white disabled:opacity-30">→
 мар {idx + 1} из {steps.length}
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 border">

```

```

const [stepIdx, setStepIdx] = useState(0);
const { l, r } = useMemo(() => splitTree(buildTree(P0),
const merged = useMemo(() => mergeTree(cloneTree(l),
const steps = merged.trace;
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;
const cur = steps[idx];

return (
 <div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 mar {i
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 g
 <div className="bg-sky-950/20 rounded-xl p-3 bo
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {cur?.chosenId ? (
 <>⊗ {cur.note}</>
) : (
 <>Готово: {done ? "деревья склеены обратно в
 />
 </>
)}
 </div>
)}
 </div>

```

```

 te-700 rounded-lg px-3 py-1.5 text-sm font-monospace
 {POINTS.map((p) => <option key={p.key} value={p.value}>{p.value}</option>)}
 </select>
 <button onClick={() => setStepIdx(Math.max(0, idx - 1))}
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(steps.length - 1, idx + 1))}
 font-bold bg-indigo-600 text-white disabled:opacity-30">
 шаг {idx + 1}
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783f0f" : "#f5f5dc")}
 nodeStroke={(id) => (pathSet.has(id) ? "#f5f5dc" : "#783f0f")}
 subtitle="Куча сама умеет удалять только вершины"
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">Валидность
 <TreeView root={res.root} subtitle="Валидность"
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сортирует по y
 каждое ребро вниз по y (куча целая) — значит,

 </div>
)}

 <div className="text-xs text-slate-300 bg-slate-900/40 rounded-xl p-4">
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-900/40 rounded-xl p-4">
 Ответ на «убрать можно только самый верхний узел» — нет.
 Ягивает merge двух детей. Удаляется любая вершина,
 </div>

```



```

 <div className="flex gap-2 overflow-x-auto mb
 {heapArr.slice(1).map((v, i) => {
 <div key={i} className="px-3 py-2 rounded
 <div className="text-slate-500 text-[10
 <div className="text-white font-bold">{
 <div className="text-emerald-400 text-[
 </div>
 </div>
 </div>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-rose-300
 >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 ro
 der-slate-700" : "bg-rose-950/40 border-ros
 <div className="text-slate-500 text-[
 {id ? <div className="text-white font
 </div>
 </div>
 </div>
 </div>
 </div>
 <p className="text-xs text-rose-300 mt-2">Смо
 то левая ветка ушла на 4 уровня и заняла да
 казателями (left/right), а не арифметикой.<
 </div>
 </div>
)}

```

```

<div className="text-xs text-slate-400 bg-slate-900/40
 Грабля «дети в массиве по $2k/2k+1$ »: фо

```

```

 }
 return steps;
 }, [sortedByKey]));

const wrongMid = Math.floor((shuffled.length - 1) / 2);
const comparisons = stage === 2 ? bidx + 1 : 0;

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти y для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsStep
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 C6poc</button>
 </div>

 <div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s :
 const isMid = stage === 2 && (() => { const s :
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} class=
 nsition-all ${highlight ? "border-rose-500 l
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px]

```

```
];
```

```
export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");
```

```
 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-emerald-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-emerald-300" >Пап
 </p>
 </div>
 </div>
 </div>
 </div>
```

```
 <div className="flex flex-wrap gap-2 mb-5" >
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 900/60 text-slate-400 hover:text-white border`
 >
 {t.label}
 </button>
)))
 </div>
```

```
{tab === "build" && <BuildMode />}
{tab === "split" && <SplitMode />}
{tab === "merge" && <MergeMode />}
{tab === "erase" && <EraseMode />}
{tab === "layers" && <LayersMode />}
{tab === "search" && <SearchMode />}
```

```

import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumViz";
import { SparseTableViz } from "../visualizers/SparseTableViz";
import { TreapBuildViz } from "../TreapBuildViz";

/** Разбор поворота и песочница всегда идут парой: посмотри
**
* Анимация и песочница идут друг под другом, а не в две
* колонке дерево сжималось до нечитаемого размера, а по
* компактности. Между ними – подпись, объясняющая перемену
*/
const SplayRotationsPair: React.FC = () => (
 <div className="space-y-4">
 <SplayRotationsViz />
 <p className="flex items-start gap-2 text-[12px] leading-3 py-2">

 Разобрался – проверь себя: ниже то же самое дерево
 не решишь.

 </p>
 <SplayRotationSandbox />
 </div>
);

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
* Единый реестр интерактивных демонстраций.
* Используется и для панели под главой, и для всплывающей
* и для модального окна «Открыть симулятор».
```

```

"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно состоит",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay-дерево",
 hint: "Покадровый разбор трёх поворотов, песочница",
 Component: () => (
 <div className="space-y-10">
 <SplayRotationsPair />
 <SplayTreeViz />
 </div>
),
},
"splay-rotations": {
 title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
 hint: "Сверху — покадровый разбор поворота, снизу —",
 Component: () => <SplayRotationsPair />,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: },
"top-sort": { title: "Топологическая сортировка", Component: },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: },
"graph-articulation": { title: "Точки сочленения", Component: },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: },
"planarity-euler-formula": { title: "Планарность: KS", Component: },
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
)
}

```

```

intro: { title: "Мнемокарточки", Component: MnemonicC
"everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
},
};
```

```
export const getViz = (id?: string): VizEntry | undefined
```

```

ФАЙЛ 69/82: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО
```

```

import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60,
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, de
```

```

 });
 const [jumpPath, setJumpPath] = useState<{ from: number; to: number; }>(null);
 const [foundMatches, setFoundMatches] = useState<{ index: number; match: string; }[]>([]);
 const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(1);

 // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
 // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
 const [trainStep, setTrainStep] = useState<number>(0);

 const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
 };

 const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
 };

 const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен!');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;

```

```

 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
 }

 if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords}
 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 },

```





```

 </div>

 {/* Переключатель режимов */}
 <div className="bg-slate-900 p-1.5 rounded-xl b
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
 </div>
 </div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В

```

```

 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 ❌ Код BFS построения fail

 <span className="text-xs bg-indigo-950 l
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>

```

```

<input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
/>
</div>

<div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map(({char, idx}) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}
 `}
 >
 {char}

)}

 </div>
 {currentIndex >= textToScan.length && (

 ГОТОВО ✓

)}
</div>

<button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
/>

```

```

 l-4 border-emerald-500 text-emerald-300 font-
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine == 4 &&
овпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold m-0">
 <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-2xl flex items-center">
 {searchLog}
 </div>
</div>
</div>
</div>
))

```

```

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
 <div className="absolute inset-0 opacity-10 bg-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

```

```

 }}}

 { /* Рендер прямых переходов (потоков воды) */
 Object.entries(TRIE_TRANSITIONS).map(([fromNode, toNode]) => {
 const fromNode = WATERFALL_NODES[Number(fromNode)];
 return Object.entries(children).map(([character, toId]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ?
 type === 'normal';

 // Подсветка в режиме обучения (когда ищем)
 let isTrainingExamined = activeMode === 'train' ?
 let isMatchBranch = isTrainingExamined & isMatchBranch;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 { /* Линия течения воды */ }
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 { /* Символ перехода */ }
 <circle cx={{(fromNode.x + toNode.x) / 2}} cy={{(fromNode.y + toNode.y) / 2}}
 r={10}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5e') : 'white'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2 + 15}}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5e') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 </g>
);
 });
 });
 }
}

```

```

const cx = node.x + dx / 2 + (node.id % 2) * dx;
const cy = node.y + dy / 2 - 30;

return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${node.x} ${node.y}`}
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
 strokeWidth={isHighlighted || isJustEstablished ? 6 : 2}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
}}
}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

return (
 <g key={`node-${node.id}`} className={isCurrent ? 'current' : ''}>
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={30} fill="none" stroke="white" stroke-width="2px" />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={30} fill="none" stroke="red" stroke-width="2px" />
)}

```

```

 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 })
</g>
);
}}}
```

{/\* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ \*/}

```

<g>
 { /* Круг-подложка под рыбу */ }
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */ }
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
</g>
```

```

</svg>
```

```

</div>
```



ФАЙЛ 70/82: `src/components/visualizers/PrefixSumViz.tsx`

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ:

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * • в 1D наводим на P[i] – подсвечивается кусок массива
 * • в 1D наводим на a[i] – видно, в какие префиксы он входит
 * • в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-mono text-sm" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto">
 {(mode === "1d" ? A1 : A2).map((arr, i) => {
```

```

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/* индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={`${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 {/* массив a */}
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'cover' })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r ? i : r))}
 className={`${cellBase} cursor-pointer`}
 inCover={inCover}
 inRange={inRange}
 style={{
 background: inCover
 ? "bg-indigo-500 text-white ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 }}
 >
 {v}
 </div>
)
 })}
 </div>
 <div>
 {a.map((v, i) => (
 <div
 key={i}
 title={`a[${i}] = ${v}`}
 style={{
 background: inCover
 ? "bg-indigo-500 text-white ring-indigo-500"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 }}
 >
 {v}
 </div>
))}
 </div>
 </div>
 </div>
 </div>
 </div>
)

```

```

{hover?.kind === "pref" ? {
 hover.i === 0 ? {
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
 } : {
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
 }
} : hover?.kind === "a" ? {
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
} : {
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
}
})
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву

```

```

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
 return (
 <div
 key={j}
 onClick={() => setRect((r) => {
 i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r;
 })}
 className={` ${cellBase} cursor-pointer ${
 inRect ? "bg-indigo-500 text-white" : ""
 }`}
 />
)
 })}
 </div>
 {v}
 </div>
)};
 </div>
 </div>
 </div>
)
);

```

```

 }}}
 </div>
 }}}
 </div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hover.j] +$
 {" "}
 – сумма всего прямоугольника от левого верхнего угла до
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на элемент
 }}
</div>

<div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-300">
 сумма = A + <span
 C + <span
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
</div>
</div>
);
};

```

```

const ST2D: number[][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d">("1d");

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

 // Total steps: we animate computing each element for each j
 const computeSteps: { i: number; j: number }[] = [];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }

 const maxStep = computeSteps.length;
 const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

 return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 </div>
 </div>
 <div>
 <table>
 <caption>Sparse Table (Min)</caption>
 <thead>
 <tr>
 <th>i</th>
 <th>j</th>
 <th>ST[i][j]</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <td>0</td>
 <td>1</td>
 <td>0</td>
 </tr>
 <tr>
 <td>1</td>
 <td>1</td>
 <td>1</td>
 </tr>
 <tr>
 <td>2</td>
 <td>1</td>
 <td>0</td>
 </tr>
 <tr>
 <td>2</td>
 <td>2</td>
 <td>1</td>
 </tr>
 <tr>
 <td>3</td>
 <td>1</td>
 <td>0</td>
 </tr>
 <tr>
 <td>3</td>
 <td>2</td>
 <td>1</td>
 </tr>
 <tr>
 <td>4</td>
 <td>1</td>
 <td>0</td>
 </tr>
 <tr>
 <td>4</td>
 <td>2</td>
 <td>1</td>
 </tr>
 <tr>
 <td>4</td>
 <td>3</td>
 <td>1</td>
 </tr>
 <tr>
 <td>5</td>
 <td>1</td>
 <td>0</td>
 </tr>
 <tr>
 <td>5</td>
 <td>2</td>
 <td>1</td>
 </tr>
 <tr>
 <td>5</td>
 <td>3</td>
 <td>1</td>
 </tr>
 <tr>
 <td>5</td>
 <td>4</td>
 <td>1</td>
 </tr>
 </tbody>
 </table>
 </div>
 </div>
);
};

```

```

 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " \}$

 min({arr1D.slice(covered.from, covered.to)}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-50">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 {/* Headers */}
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>
 </div>
 </div>
</div>

```



```

 if {currentStep.i + (1 << (current
 isSrc2 = true;
 }
 }

 return (
 <div key={j} className="relative fl
 {!isValid ? (
 <div className="w-[clamp(30px,9
) : (
 <motion.div
 onMouseEnter={() => isVisible
 onMouseLeave={() => setHover(
 onClick={() => isVisible && s
 initial={{ opacity: 0, scale:
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-r
 : isSrc2
 ? "bg-amber-500 text-r
 : isVisible
 ? "bg-slate-800 tex
 : "bg-transparent t
 }`}
 >
 {st1D[i][j]}
 </motion.div>
)}
)}

```

```

 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-b"
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo"
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b"
 {st1D[c.i][c.j]}

 </p>
);
 })(()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: num

```

```

<div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
}>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
</h4>
 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
 let zIndex = 'z-0';

 const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft
 highlightClass = 'bg-blur
] md:scale-[1.08]';

```



```

 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}{()}}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
 }}
</div>
 }}
</div>
</div>
</div>
);
};

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

```

```

top: `${(r1 / 8) * 100}%`,
left: `${(c1 / 8) * 100}%`,
marginTop: '8px',
marginLeft: '8px',
height: 'calc(' + (H/8*100) +
width: 'calc(' + (W/8*100) +
 }}
/>

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
 ${Ly / 8} * 100}% - 16px)`, height: `calc(
 ${H / 8} * 100}% - 16px)`, width: `calc(
 ${Lx / 8} * 100}% - 16px)`, left: `calc(
 ${Lx / 8} * 100}% - 16px)`, right: `calc(
 ${Lx / 8} * 100}% - 16px)`, bottom: `calc(
 ${Ly / 8} * 100}% - 16px)`,
 <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
 ${Ly / 8} * 100}% - 16px)`, height: `calc(
 ${H / 8} * 100}% - 16px)`, width: `calc(
 ${Lx / 8} * 100}% - 16px)`, left: `calc(
 ${Lx / 8} * 100}% - 16px)`, right: `calc(
 ${Lx / 8} * 100}% - 16px)`, bottom: `calc(
 ${Ly / 8} * 100}% - 16px)`,
 <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(
 ${Ly / 8} * 100}% - 16px)`, height: `calc(
 ${H / 8} * 100}% - 16px)`, width: `calc(
 ${Lx / 8} * 100}% - 16px)`, left: `calc(
 ${Lx / 8} * 100}% - 16px)`, right: `calc(
 ${Lx / 8} * 100}% - 16px)`, bottom: `calc(
 ${Ly / 8} * 100}% - 16px)`,
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
 ${Ly / 8} * 100}% - 16px)`, height: `calc(
 ${H / 8} * 100}% - 16px)`, width: `calc(
 ${Lx / 8} * 100}% - 16px)`, left: `calc(
 ${Lx / 8} * 100}% - 16px)`, right: `calc(
 ${Lx / 8} * 100}% - 16px)`, bottom: `calc(
 ${Ly / 8} * 100}% - 16px)`,
 </div>
 </div>

 <div className="bg-slate-950 p-4 rounded-2xl shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{ width: 100%, height: 100% }}>
 <label className="flex items-center justify-between" style={{ width: 100% }}>

 <div className="flex gap-2" style={{ width: 100% }}>
 <input type="number" min={0} max={100} value={r1} style={{ width: 50px; height: 30px; border: 1px solid #333; background-color: #f0f0f0; }} />
 <input type="number" min={0} max={100} value={c1} style={{ width: 50px; height: 30px; border: 1px solid #333; background-color: #f0f0f0; }} />
 </div>

 </label>
 <label className="flex items-center justify-between" style={{ width: 100% }}>

 <div className="flex gap-2" style={{ width: 100% }}>
 <input type="number" min={0} max={100} value={r2} style={{ width: 50px; height: 30px; border: 1px solid #333; background-color: #f0f0f0; }} />
 <input type="number" min={0} max={100} value={c2} style={{ width: 50px; height: 30px; border: 1px solid #333; background-color: #f0f0f0; }} />
 </div>

 </label>
 </div>

```

```


В запроса)

r1][

во, чтобы правый край уперся в c2)

r1][

px, чтобы нижний край уперся в r2)

nded">r2 - Lx + 1][

и влево от c2)

">r2 - Lx + 1][
 {'}}''));
</div>

<div className="w-full flex justify-center">
 <div className="flex flex-col items-center">
 <h4 className="text-slate-400 font-bold">
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
 </h4>
 <div className="grid gap-[2px] p-2.5">
Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;
 })}
 </div>
 </div>
</div>

```

```


 + 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly]
 </div>
 </div>
 </div>
 </div>
)
 }
}
```

---

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

ФАЙЛ 72/82: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 137 | РАЗМЕР

---

```

 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold text-emerald-500'
 : 'px-3 py-1 rounded text-sm font-bold text-gray-500'

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold text-emerald-500'
 : 'px-3 py-1 rounded text-sm font-bold text-gray-500'

 const aside = document.querySelector('aside')
 const headerMain = document.querySelector('h1')
 const questionsContainer = document.getElementById('questions-container')
```



```

 n-colors';
 document.getElementById('btn-theme-light').onmouseover = function() {
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 };
 document.getElementById('btn-theme-notebook').onmouseover = function() {
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 };
 }

 // Initialize theme and mode
 setTheme('dark');

 // Setup intersection observer for TOC highlighting
 document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('hidden');
 if (isClosed) {
 mobileDrawer.classList.remove('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = 'hidden';
 } else {
 mobileDrawer.classList.add('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
 }
 document.body.style.overflow = '';
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
 });

```

```
 }
 });
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
 });
</script>
</body>
</html>
```

---

ФАЙЛ 73/82: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "../src/index.css";
```

```

 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
 }
 mjx-container[display="true"]::-webkit-scrollbar {
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 12px; }
 .uno-card::before, .uno-card::after { font-size: 12px; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 40px !important; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 10px; }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in containers */
 .formula-scroll { flex-wrap: nowrap !important; }
 }

 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }

 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; } .card-black::before, .card-black::after { text-

```

```

 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
 }
 body.theme-notebook .text-slate-200, body.theme
 ant; }
 body.theme-notebook .text-slate-400 { color: #5
 body.theme-notebook .border-slate-600, body.the
 : #94a3b8 !important; }
 body.theme-notebook .font-mono { font-family: "
 body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
 <div class="sticky top-0 z-50 w-full bg-slate-950/9
 center items-center gap-4 sm:gap-8 transition-col

```

```

 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf"
</marker>
<marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63"
</marker>
<marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8"
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="20"
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#33
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f

```

```
 </h3>
 <button id="close-drawer-btn" class="
 <svg xmlns="http://www.w3.org/20
 <path stroke-linecap="round
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium
 <details class="group" open>
 <summary class="list-none cursor

 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 1.1. Группы
 1.2. Кольца
 1.3. Поля
 1.4. Модули
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-emerald-blue-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 2.1. Комплексные числа
 2.2. Комплексные функции
 2.3. Комплексные интегралы
 </div>
 </details>
```

```

 <a href="#q5-def" class="block ho
 <a href="#q5-group" class="bloc
 <a href="#q5-prim" class="block
 <a href="#q5-crit" class="block
 <a href="#q5-cyclic" class="blo
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q6-1" class="block ho
 <a href="#q6-2" class="block ho
 <a href="#q6-3" class="block ho
 <a href="#q6-4" class="block ho
 <a href="#q6-5" class="block ho
 <a href="#q6-6" class="block ho
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
```

```
<summary class="list-none cursor-pointer-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

</summary>
<div class="pl-6 space-y-2 text-rose-400">

a>

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочленов

 </summary>
 <div class="pl-6 space-y-2 text-cyan-400">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-blue-500 pl-3 font-bold text-blue-400">
```



```
 border border-rose-500/30"> К
 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-teal-500 font-medium">

 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-teal-500 font-medium">

 16. Неприводимые многоч

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>
```

```
 <div class="mt-4 mb-2 text-xs font-sans text-teal-500 pl-3">


```

```

 });

 // Scroll spy logic
 document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 'h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16, h17, h18, h19, h20, h21, h22, h23, h24, h25, h26, h27, h28, h29, h30, h31, h32, h33, h34, h35, h36, h37, h38, h39, h40, h41, h42, h43, h44, h45, h46, h47, h48, h49, h50, h51, h52, h53, h54, h55, h56, h57, h58, h59, h60, h61, h62, h63, h64, h65, h66, h67, h68, h69, h70, h71, h72, h73, h74, h75, h76, h77, h78, h79, h80, h81, h82, h83, h84, h85, h86, h87, h88, h89, h90, h91, h92, h93, h94, h95, h96, h97, h98, h99, h100'
);
 const divs = document.querySelectorAll(
 'div[id^="q5-"], div[id^="q6-"], div[id^="q7-"], div[id^="q8-"], div[id^="q9-"], div[id^="q10-"], div[id^="q11-"], div[id^="q12-"], div[id^="q13-"], div[id^="q14-"], div[id^="q15-"], div[id^="q16-"], div[id^="q17-"], div[id^="q18-"], div[id^="q19-"], div[id^="q20-"], div[id^="q21-"], div[id^="q22-"], div[id^="q23-"], div[id^="q24-"], div[id^="q25-"], div[id^="q26-"], div[id^="q27-"], div[id^="q28-"], div[id^="q29-"], div[id^="q30-"], div[id^="q31-"], div[id^="q32-"], div[id^="q33-"], div[id^="q34-"], div[id^="q35-"], div[id^="q36-"], div[id^="q37-"], div[id^="q38-"], div[id^="q39-"], div[id^="q40-"], div[id^="q41-"], div[id^="q42-"], div[id^="q43-"], div[id^="q44-"], div[id^="q45-"], div[id^="q46-"], div[id^="q47-"], div[id^="q48-"], div[id^="q49-"], div[id^="q50-"], div[id^="q51-"], div[id^="q52-"], div[id^="q53-"], div[id^="q54-"], div[id^="q55-"], div[id^="q56-"], div[id^="q57-"], div[id^="q58-"], div[id^="q59-"], div[id^="q60-"], div[id^="q61-"], div[id^="q62-"], div[id^="q63-"], div[id^="q64-"], div[id^="q65-"], div[id^="q66-"], div[id^="q67-"], div[id^="q68-"], div[id^="q69-"], div[id^="q70-"], div[id^="q71-"], div[id^="q72-"], div[id^="q73-"], div[id^="q74-"], div[id^="q75-"], div[id^="q76-"], div[id^="q77-"], div[id^="q78-"], div[id^="q79-"], div[id^="q80-"], div[id^="q81-"], div[id^="q82-"], div[id^="q83-"], div[id^="q84-"], div[id^="q85-"], div[id^="q86-"], div[id^="q87-"], div[id^="q88-"], div[id^="q89-"], div[id^="q90-"], div[id^="q91-"], div[id^="q92-"], div[id^="q93-"], div[id^="q94-"], div[id^="q95-"], div[id^="q96-"], div[id^="q97-"], div[id^="q98-"], div[id^="q99-"], div[id^="q100-"]'
);
 let isScrolling = false;

 document.querySelectorAll('.group-link').forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 1000);
 });
 });

 const observer = new IntersectionObserver((entries) => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingLink = document.querySelector(`a[href="#${visibleId}"]`);
 if (correspondingLink) {
 const details = correspondingLink.closest('details');
 if (details && !details.open) {
 details.open = true;
 }
 }
 });
 });

```

```
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в ' + format;
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей браузеров, для загрузки файла необходимо открыть эту страницу в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
```

```

// Smart markdown parsing based
clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
});
clone.querySelectorAll('.img-ca

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
 fallbackModal(text, 'Markdow
 return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_alge
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate
);

```

```

 or: #94a3b8 !important; }
 body.bg-white .img-capt

 /* Tweak card blocks */
 body.bg-white .card-red

 body.bg-white aside { b
#334155 !important; }
 body.bg-white .text-whi

 /* Ensure active links
 body.bg-white aside a.t
 body.bg-white aside a.t
 body.bg-white aside a.t
 body.bg-white aside a.t
 body.bg-white aside a.t
 body.bg-white aside a.t
 body.bg-white aside a:h

 body.bg-white table tr.
 body.bg-white table tr.
 body.bg-white .text-tea
 body.bg-white .text-ros
 body.bg-white .text-eme
 body.bg-white .text-cya
 body.bg-white .text-amb
 body.bg-white .text-fuc
 body.bg-white .text-lim
 body.bg-white .text-blur
 body.bg-white .text-gre
 body.bg-white .text-ind

 `;
 } else if (theme === 'terminal'
 body.classList.add('bg-black
 document.getElementById('th
 style.innerHTML = `
 body.bg-black .bg-slate

```

```

 </button>
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-green-500
терминала"> </button>
 </div>
 </div>
 </div>
</div>

</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 <!-- Print Table of Contents -->

```

```

 <div class="bg-orange-900/40 border-4 border-yellow-500">
ольцо полиномов</div>
 <div class="bg-cyan-900/40 border-4 border-yellow-500">
сые числа</div>

 <!-- Row 1: База -->

-4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-400">База
 <div class="font-bold text-slate-200">База

-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-400">База
 <div class="font-bold text-slate-200">База

-4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500">База
 <div class="font-bold text-slate-200">База

 <!-- Row 2: Базовые операции / Евклид -->

-4 border-yellow-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-yellow-400">Базовые операции
 <div class="font-bold text-slate-200">Базовые операции

-1-4 border-orange-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-orange-400">Базовые операции
 <div class="font-bold text-slate-200">Базовые операции

-4 border-cyan-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-cyan-500">Базовые операции
 <div class="font-bold text-slate-200">Базовые операции


```

```


 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1"
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setM
-1-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-500"
 <div class="font-bold text-slate-700"

 <a href="#question-18" onclick="setM
-1-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-500"
 <div class="font-bold text-slate-700"

```



```
<div class="bg-red-500/20 p-3 m-10">
 <svg class="w-8 h-8 text-red-500"
 rg/2000/svg">
 <path stroke-linecap="round"
 110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border"
</div>
```

```
<div class="prose prose-invert max-w-4xl">
 <p class="text-lg">
 Хватит тупить. Доказательств
 есь для того, чтобы жать на кнопки. Вот жес
 </p>
```

```
<h2 class="text-2xl font-bold b"
(Меметика)</h2>
```

```
<!-- Прямое -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-blue-400">
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5 sp"
 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
ово.

</div>
```

```
<!-- От противного -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-fuch"
 <p class="text-sm text-slate"
 <ul class="list-disc pl-5 sp"
 <span class="text-w
```

```
<p class="text-sm text-slate-500">
 <ul class="list-disc pl-5 space-y-2">
 <span class="text-w"
вверх по уравнениям).
 <span class="text-w"
рху вниз, показывая, что $\delta \vdots r_i$
 <span class="text-w"
х. Твой d больше.

</div>
```

```
<h2 class="text-2xl font-bold text-slate-500">
 алгоритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2">
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-500">
 <p class="text-xs font-slate-500">
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-500">
 <p class="text-xs font-slate-500">
 c . Пока получается 0 – корень жив. Как то
корня минус один.</p>
</div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-500">
 <p class="text-xs font-slate-500">
ициентов (они делятся на p), кроме старше
зложить.</p>
</div>
```

```
 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 есть сложение — есть ноль. Если есть умножение, то
 <code class="text-x"
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 авь их драться. Умножение всегда врывается и
 <code class="text-x"
 </div>

 </div>
 </section>
 </div>

 <div id="questions-container">
 <!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 74/82: src/utills/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```

 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 }
 return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding
.export-head { border-bottom: 1px solid #1e293b; padding
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; fon
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; fon
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; f
.export-foot { margin-top: 3rem; border-top: 1px solid
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important
 pre { white-space: pre-wrap; }
}
`;

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replac

```



```

export async function downloadChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}
```

/\*\* Выгрузить всё пособие одним файлом со сквозным оглавлением \*/

```
export async function downloadBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a></li>`)
 .join("\n");

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="border: 1px solid #0f172a; padding: 1rem 1.5rem 0 1.5rem; margin-bottom: 1rem">
 <p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">
 <small>Глава ${c.id} из ${chapters.length}</small>
 </p>
 ${body}
 </div>`);
 });

 const body = `<nav id="toc" style="background:#0f172a;color:#fff;padding:1rem 1.5rem 0 1.5rem">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0">Оглавление
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem">
 1. Введение

 2. Основы

 3. Практика

 4. Заключение

 </nav>
 ${bodies.join("")}
 <script>
 <!-- CSS -->
 <style>
 @import url("${css}");
 </style>
 </script>
 </div>`;
```

```

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")]
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy &&

```

```
* Проверка покрытия глав всплывающими подсказками:
* в каждой ли теме находятся термины из глоссария.
*
* node scripts/audit-terms.mjs
*/
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
 external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => esc(l.label))})`
 "giu"
);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
 .replace(/<[^\>]+>/g, " ");
```



```

* Склеивает отрендеренные страницы tmp/export-pages/pages/ в
* в единый документ public/export/full_code_all_pages.pdf
*/
```

```
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,
```

```
// A4 в пунктах PDF
```

```
const A4 = { width: 595.28, height: 841.89 };
```

```
async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }
```

```
 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();
```

```
 if (pages.length === 0) throw new Error("Не найдено ни одной страницы");
```

```
 ensureDir(OUT_DIR);
```

```
 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });
```

```
 const stream = fs.createWriteStream(PDF_PATH);
```

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и си
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+\$/ , "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
```

```

 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
}

/** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
 }));
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

```

```

 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
 });

 push(HR);
 push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
 push(HR);

 const txt = out.join("\n") + "\n";
 fs.writeFileSync(TXT_PATH, txt, "utf8");

 console.log("[1/3] код -> txt");
 console.log(`файлов: ${files.length}`);
 console.log(`глав: ${chapters.length}`);
 console.log(`строк: ${txt.split("\n").length}`);
 console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

```
 rows: 76,
 };

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
);
}
```

---

ФАЙЛ 81/82: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.5 KB

---

/\*\*

```

 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
 }

 async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 });стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "serif",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
];

```

```
});
```

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

ФАЙЛ 82/82: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.1 КБ

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (artifacts)
|
→ full_code_all_page.pdf (artifacts)
#
Готовые TXT и PDF коммитаются обратно в ветку (их отдаёт GitHub)
промежуточные PNG сохраняются как artifacts запуска.
```

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бесконечно

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умолчанию 300)"

required: false

```
 fi
 done
 convert -version

- name: Install dependencies
 run: npm ci --no-audit --no-fund

- name: Step 1 – код → txt
 run: npm run export:txt

- name: Step 2 – txt → png
 env:
 EXPORT_DENSITY: ${GITHUB_EVENT_INPUTS.density}
 run: npm run export:png

- name: Step 3 – png → pdf
 run: npm run export:pdf

- name: Type-check приложения
 run: npx tsc --noEmit
 continue-on-error: true

- name: Summary
 run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`${GITHUB_WORKSPACE}/public/export/full_code_all_pages\` | $(du -h --max-depth=1 --exclude=public/export/full_code_all_pages | grep '\.txt$' | sort -n | tail -n 1) |"
 echo "| \`${GITHUB_WORKSPACE}/public/export/full_code_all_pages\` | $(du -h --max-depth=1 --exclude=public/export/full_code_all_pages | grep '\.png$' | sort -n | tail -n 1) |"
 echo "| PNG-страниц | $(ls tmp/export-pages | wc -l) |"
 } >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
 uses: actions/upload-artifact@v4
 with:
 name: full-code-export
```